

Department of Industrial Engineering and Production

Master of Research

" Data Science for Smart Manufacturing "

Research Project Title

Community Detection in Social Networks for
Targeted Marketing and Recommender Systems

Presented by:

MOUDDEN Hamza

ENSAM ID: FM24005

Supervised by:

Pr. ELYAZID Akachar

Academic Year: 2025-2026

Acknowledgements

First and foremost, I thank Almighty God for granting me the strength, patience, and determination necessary to complete this work.

This final year's project would not have been possible without the support and valuable contribution of several people, to whom I wish to express my deep gratitude.

I would like to warmly thank my supervisor, Professor ELYAZID Akachar, for agreeing to guide this work. I am extremely grateful for his availability, insightful advice, patience, and the trust he placed in me throughout this journey. His relevant guidance and scientific rigor have greatly contributed to the improvement of this work and to my personal development.

My thanks also go to the jury members for taking the time to evaluate this work. Their remarks and suggestions will be valuable in enriching this research and opening new perspectives.

I wish to express my gratitude to all the professors of the Master's program in Data Science for Smart Manufacturing. Their dedication, the quality of their teaching, and their support throughout my training have been decisive in the success of my academic journey.

A great thank you to my parents, for their unconditional love, their sacrifices, and their unwavering support. Their belief in me has been my driving force during difficult times. This work is the result of their continuous support and encouragement.

I do not forget my friends and fellow classmates, with whom I shared unforgettable moments, enriching discussions, and valuable mutual support. Their friendship has made this journey more enjoyable and stimulating.

Table of Contents

<i>Acknowledgements</i>	2
Chapter 1: Introduction and Problem Statement.....	6
Introduction.....	6
Background	7
Problem Statement.....	7
Objectives	8
Chapter 2: State of the Art.....	10
Introduction to Chapter.....	10
Graph Theory Fundamentals	10
Community Detection in Graphs	14
Classical Community Detection Algorithms.....	18
Modern Approaches.....	27
Applications of Community Detection	34
Standard Benchmark Datasets for Community Detection.....	36
LFR Benchmark	40
Foundations of Recommender Systems	41
Community Detection for Recommender Systems.....	42
Conclusion of Chapter	44
Chapter 3: Implementation and Experimental Results	46
Introduction to Chapter.....	46
Experimental Setup.....	46
Classical Algorithms Results on Real-World Datasets	48
Results of Deep Learning Methods	50
LFR Benchmark Results	51
Recommender System Integration	54
Chapter 4: Conclusion and Future Perspectives	65
Summary of Contributions	65

Table of Figures

Figure 1 : Undirected vs Directed Graphs	11
Figure 2 : Weighted Graphs.....	11
Figure 3 : Adjacency Matrix.....	12
Figure 4 : Betweenness concept	13
Figure 5 : Connected components of an undirected graph.....	13
Figure 6 : Dense vs Sparse Graphs	14
Figure 7 : Balanced vs Real world Partition	16
Figure 8 : Overlapping communities	16
Figure 9 : Louvain Algorithm.....	18
Figure 10 : Leiden Algorithm	19
Figure 11 : Girvan Newman Algorithm	22
Figure 12 : LPA Algorithm.....	24
Figure 13 : Illustration of the graph embedding pipeline from knowledge graph to feature vectors.....	28
Figure 14 : Architecture of a Multi-layer Graph Neural Network.	30
Figure 15 : Schematic representation of GCN layers.	31
Figure 16 : Graph Attention Network	32
Figure 17 : Architecture of a VGAE with encoder and decoder	33
Figure 18 : Original vs Coarsened Graphs	35
Figure 19 : Semantic-topological community detection framework.....	36
Figure 20 : Zachary's Karate Club	37
Figure 21 : Impact of the parameter u on the detectability of clusters in heterogeneous networks.	40
Figure 22 : Overview of the CBMF recommendation pipeline.	42
Figure 23 : Schematic representation of the hybrid food recommendation model proposed by Rostami et al. [34].	43
Figure 24 : LFR Benchmark Results: Network Size = 500 nodes	51
Figure 25 : LFR Benchmark Results: Network Size = 1000 nodes	51
Figure 26 : LFR Benchmark Results: Network Size = 5000 nodes	52
Figure 27 : LFR Benchmark Results: mixing parameter = 0.7	52
Figure 28 : LFR Benchmark Results: Network Size = 10000 nodes	53
Figure 29 : LFR Benchmark Results: Network Size = 50000 nodes	53
Figure 30 : LFR Benchmark Results: Network Size = 100000 nodes	54
Figure 31 : User-Based Recommender.....	55
Figure 32 : Item-Based Recommender	56
Figure 33 : SVD of a matrix.....	57
Figure 34 : Alternating Least Squares Method	58
Figure 35 : Best Matching 25 Algorithm.....	59
Figure 36 : Comparative performance of all evaluated recommender approaches	60
Figure 37 : SVD-based and ALS-based hybrid recommenders versus pure community baselines.....	61
Figure 38 : Precision-recall scatter plot illustrating trade-offs across different method categories	61
Figure 39 : Performance comparison across different community detection algorithms for each hybrid type.....	62
Figure 40 : Relative percentage improvement of SVD and ALS hybrid methods over the best pure community baseline	62

Chapter 1: Introduction and Problem Statement

Chapter 1: Introduction and Problem Statement

Introduction

Social media platforms have experienced explosive growth over the past decade. Billions of individuals around the world now use services such as Facebook, Twitter, Instagram, and Reddit daily. These platforms generate enormous volumes of data every single moment. This data includes various forms of user activity, such as liking posts, sharing content, leaving comments, and following other users. When examined together, these actions form intricate networks that reveal important patterns about how people connect and interact with one another.

Researchers and businesses alike have shown strong interest in understanding the hidden structures within these online social networks. One particularly important type of structure is the presence of groups where members are more strongly connected to each other than to people outside the group. These clusters are commonly referred to as communities in network science. The task of automatically identifying these groups is known as community detection, and it represents a fundamental challenge in the analysis of complex networks.

This work applies community detection techniques to graphs derived from social media. The goal is to use the discovered communities for two practical purposes. The first purpose is targeted marketing, which involves delivering advertisements to specific groups of users who are likely to be interested in a product or service. The second purpose is recommender systems, which suggest items, content, or connections that a user may find appealing.

By identifying communities of users who share common interests or behavioral patterns, marketers can move away from broadcasting the same message to everyone. Instead, they can tailor their campaigns to resonate with specific communities. Similarly, recommendation algorithms can take advantage of community information to provide more personalized suggestions. For example, a user might receive recommendations based on what other members of their community have enjoyed.

The combination of community detection with marketing and recommendation offers exciting possibilities for building intelligent, data-informed applications. However, several obstacles remain. These challenges include using the most appropriate algorithms, measuring the quality of the

communities that are found, and effectively incorporating community information into real-world systems.

Background

Social media platforms can be modelled as graphs, where users are represented as nodes and interactions between them are represented as edges. These graphs are typically very large and sparse, meaning that only a small fraction of possible connections exist.

Two important characteristics are often observed in such networks. The first is the small-world property, which indicates that most users can be reached from one another through a small number of steps. The second is the scale-free nature of these networks, where a few nodes have a very high number of connections while the majority have relatively few.

Community detection algorithms aim to divide the network into groups such that connections within the same group are dense, while connections between groups are limited. These communities often reflect real-world groupings such as shared interests or social circles.

This concept has practical importance. In marketing, identifying communities helps in targeting specific audiences and identifying influential users. In recommendation systems, it can improve personalization and help overcome issues like lack of data for new users or limited interaction history.

Problem Statement

Despite the large number of community detection algorithms available in the literature, selecting the most appropriate method for real-world applications remains a challenging task.

First, existing approaches differ significantly in their underlying principles, ranging from modularity optimization and random walks to embedding-based and deep learning methods. This diversity makes it difficult to determine which techniques are best suited for specific use cases.

Second, most studies evaluate these algorithms using structural metrics such as modularity, which measure how well a network is partitioned. However, these metrics do not necessarily reflect the practical usefulness of the detected communities in applications such as marketing or recommendation systems.

Third, while modern approaches like graph embeddings and graph neural networks have shown promising results, their added value compared to classical methods is not always clearly demonstrated, especially in large-scale social network scenarios.

Finally, there is a lack of comparative analysis that considers both traditional and modern techniques within a unified framework, particularly when evaluated on realistic datasets and application-oriented criteria.

Based on these limitations, the main research question of this project is:

How do different community detection approaches, ranging from classical algorithms to modern graph learning methods, compare in terms of their effectiveness and practical utility for recommender systems?

Objectives

The main objectives of this project can be summarized as follows:

- To study and categorize community detection approaches, including both classical algorithms and modern graph-based learning methods
- To implement a selection of representative algorithms from different categories (e.g., Louvain, Label Propagation, and embedding-based methods)
- To apply these algorithms to real-world and synthetic network datasets
- To evaluate the detected communities using structural metrics such as modularity and conductance
- To assess the usefulness of these communities in practical applications, particularly targeted marketing and recommendation systems
- To compare the performance of different approaches and identify their strengths and limitations
- To provide insights into the trade-offs between simplicity, scalability, and effectiveness

Chapter 2: State of the Art

Chapter 2: State of the Art

Introduction to Chapter

This chapter reviews the existing literature on community detection and its applications in recommender systems. It begins with fundamental concepts from graph theory, including graph types, centrality measures, and network properties. The chapter then defines what constitutes a community in network science and discusses the challenges of evaluating community quality. A comprehensive survey of classical community detection algorithms is presented, followed by modern deep learning approaches. The chapter also covers benchmark datasets used for evaluation and concludes with an overview of recommender systems and how community detection has been applied to address their fundamental challenges.

Graph Theory Fundamentals

Introduction

Graph theory provides the mathematical framework used to model and analyze complex systems such as social networks, biological systems, and information networks. In this context, a graph is defined as a structure composed of a set of nodes (vertices) connected by edges that represent relationships between entities.

Formally, a graph is denoted as $G = (V, E)$, where V is the set of nodes and $E \subseteq V \times V$ is the set of edges. Graph-based representations are widely used in network analysis because they allow capturing both the structure and interactions within a system [1].

Directed and Undirected Graphs

Graphs usually fall into two categories based on the kind of relationships they show: undirected or directed.

In undirected graphs, edges have no direction, so the relationship is symmetric. For instance, if you're modelling a friendship where both people feel the same way, an undirected graph works perfectly.

Directed graphs or digraphs take things a step further by giving each edge a direction. Here, an edge from u to v isn't the same as one from v to u . You see this a lot on social media, where following someone doesn't mean they follow you back.

One big advantage of directed graphs is that they let you track incoming links (in-degree) and outgoing links (out-degree) separately, which tells you a lot more about each node's role.

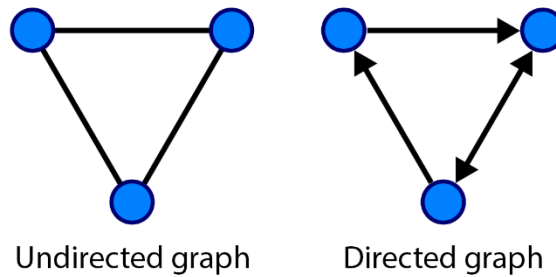


Figure 1 : Undirected vs Directed Graphs

Weighted Graphs

In a lot of real-world situations, not every connection between nodes carries the same meaning. That's where weighted graphs come in, they're basically an extension that adds a numerical value, or weight, to each edge.

That weight usually reflects something like strength, frequency, or significance of interaction. Take a social network, for instance: you might use weights to capture how many messages two people exchange, the number of likes or comments between them, or how often they interact.

Compared to unweighted graphs, weighted ones give you a much richer picture. They let algorithms consider not just whether a connection exists, but how strong it is [2].

The trade-off, though, is that adding weight makes things more computationally demanding. You also must be careful about normalizing the values depending on what algorithm you're using.

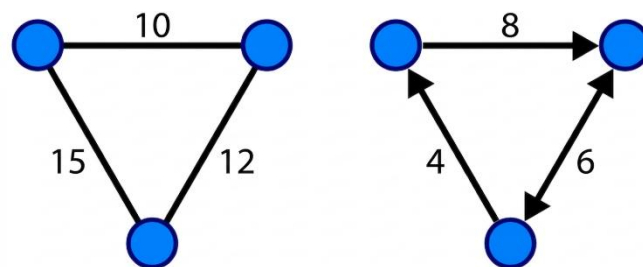


Figure 2 : Weighted Graphs

Adjacency Matrix

The adjacency matrix is a go-to mathematical way to represent graphs. For a graph with n nodes, it's an $n \times n$ matrix A where each A_{ij} tells you something about the connection between node i and node j .

In an unweighted graph:

- $A_{ij} = 1$ if an edge exists between i and j
- $A_{ij} = 0$ otherwise

For weighted graphs, A_{ij} simply stores the weight of that edge.

Undirected graphs give you a symmetric adjacency matrix, whereas directed ones usually don't.

This representation shows up all over the place in practice especially in algorithms like spectral clustering, graph convolutional networks (GCNs), and a wide range of network analysis tasks.

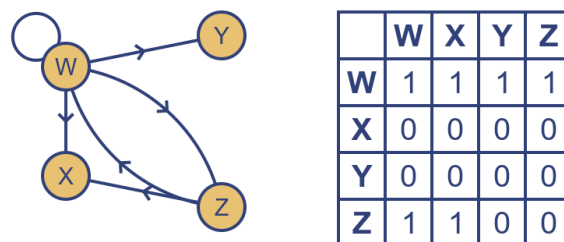


Figure 3 : Adjacency Matrix

Degree and Centrality Measures

One of the simplest but most powerful graph measures is node degree. It's just the number of edges connected to a node.

In undirected graphs, you get a single degree value. In directed graphs, there's a split: in-degree for incoming connections and out-degree for outgoing ones.

High-degree nodes are often seen as important or influential.

But if you want a deeper look at node importance, there are a few go-to centrality measures:

- Degree Centrality – basically just the degree itself; measures direct connections.

- Betweenness Centrality – tracks how often a node lies on shortest paths between others. High betweenness means it's acting as a bridge.
- Closeness Centrality – measures how quickly a node can reach everyone else in the network.

These are especially handy for things like spotting influencers, modeling how information spreads, and designing marketing strategies [3].

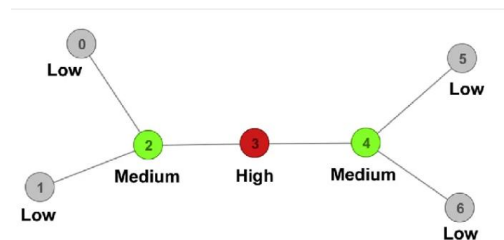


Figure 4 : Betweenness concept

Paths and Connected Components

In graph terms, a path is just a sequence of nodes where each pair in order is connected by an edge. Paths are essential for understanding how things travel through a network.

When there's a path between every pair of nodes, the graph is connected. If not, it splits into connected components subgraphs where nodes are all reachable from one another but not to nodes in other components.

Take real-world networks like social platforms: you usually have one giant connected component holding most of the users, plus a handful of smaller isolated components.

Connectivity isn't just a theoretical concept, it's key for community detection, analyzing how resilient a network is, and modeling how information propagates.

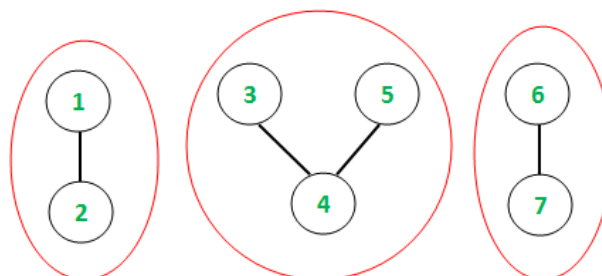


Figure 5 : Connected components of an undirected graph

Graph Density

Graph density is a simple way to describe how many edges a graph has relative to the maximum possible.

For undirected graphs, you calculate it as:

$$\text{Density} = \frac{2 | E |}{| V | (| V | - 1)}$$

Dense graphs have lots of edges; sparse graphs have relatively few.

Most real-world networks, especially social networks fall into the sparse category. Typically, each node connects to only a small fraction of the other nodes.

This sparsity isn't just an observation it has real implications. It reduces computational costs and makes community detection more practical and meaningful.

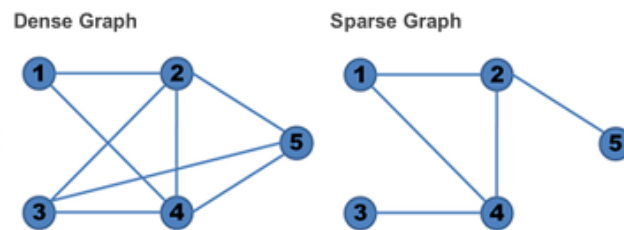


Figure 6 : Dense vs Sparse Graphs

Community Detection in Graphs

Definition of a Community

Community is a central concept in network science, but there's no universally accepted definition. Fortunato describes it broadly as a group of nodes that are more densely connected internally than externally [4]. That captures the basic intuition communities are meaningful substructures, whether these are social circles on online platforms or functional modules in biological networks.

Fortunato notes that this ambiguity comes from the diversity of real-world networks. What counts as dense or significant varies, so different algorithms often produce different results on the same network [4]. Later research reinforces this, framing community detection as inherently ill-posed and context-dependent [5], [6].

Newer approaches have pushed beyond structural definitions. Probabilistic models define communities by shared connection patterns, while embedding-based methods locate communities in latent spaces where structurally similar nodes sit closer together [7], [8]. Together, these reflect a broader evolution from purely graph-theory definitions toward more data-driven representations.

Local and Global Definitions of Communities

Community definitions generally split into local and global approaches.

Local definitions focus on subgraph structure. A set of nodes counts as a community if it meets local properties like high internal density or a strong clustering coefficient. These methods are fast computationally but can overlook large-scale network organization [4].

Global definitions consider the whole network and optimize an overall objective. Modularity is a prime example: it compares edge density inside communities to what you'd see in a random graph [4], [9].

According to Fortunato, global methods tend to give more consistent, meaningful partitions, but they come with heavy computational costs [4]. Recent advances address this through scalable approximations and parallel algorithms [10].

Modern approaches also incorporate machine learning, letting us infer communities from both structure and node attributes. These hybrid methods offer more flexible and expressive community representations [7], [11].

Graph Partitioning Perspective

Community detection can also be framed as a graph partitioning problem: divide the network so that connections inside each group are maximized and connections between groups are minimized [4].

Traditional graph partitioning comes with strict constraints like requiring a predefined number of clusters and forcing them to be balanced in size.

But real-world networks rarely fit that mold. Social networks, for example, have skewed degree distributions and highly uneven community sizes, so classical methods often fall short [5].

Modern approaches address this by supporting:

- Flexible community sizes
- Hierarchical structures
- More adaptable partitioning criteria

These advances have made community detection far more applicable to real-world scenarios [6], [10].

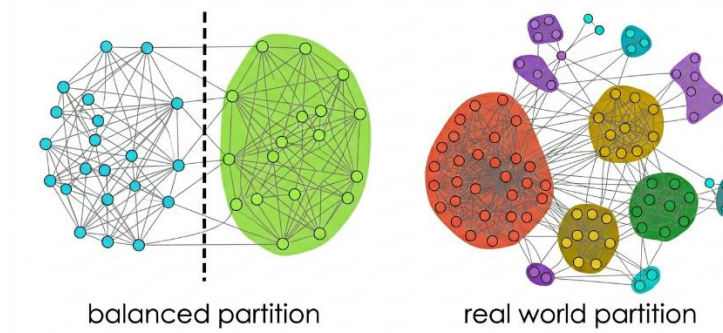


Figure 7 : Balanced vs Real world Partition

Disjoint and Overlapping Communities

Community structures can also be categorized by how membership works.

In disjoint communities, each node belongs to exactly one group. This keeps things simple and is common in classical algorithms, but it often oversimplifies real-world complexity.

Overlapping communities allow nodes to belong to multiple groups at once. That's especially useful for social networks, where people are part of different circles like family, work, or hobbies [5], [12].

Fortunato discusses overlapping detection techniques such as clique percolation, which finds communities by linking fully connected subgraphs into chains [4].

Studies show that overlapping community detection leads to more realistic representations and better results in areas like recommender systems and social network analysis [11], [13].

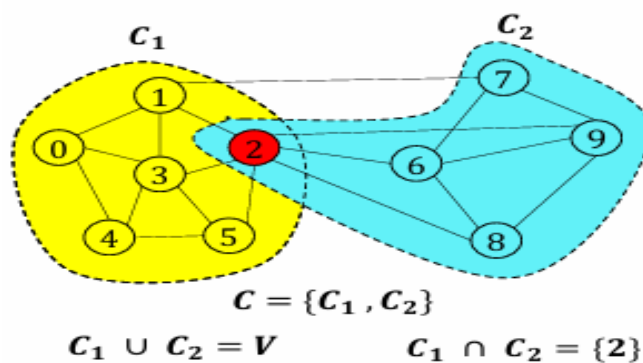


Figure 8 : Overlapping communities

Quality Functions

Evaluating community quality is a crucial part of community detection. The go-to measure here is modularity, developed by Newman and Girvan.

Modularity calculates the difference between the actual number of internal edges and what you'd expect in a random graph with the same degree distribution [9]. High modularity signals strong community structure.

But modularity has its flaws. Fortunato highlights that it struggles to detect small communities in large networks [4]. This led to the development of alternative metrics and multi-resolution methods.

Recent studies propose improved metrics that overcome these limitations, including multi-scale modularity, stability-based measures, and likelihood-based evaluation methods [6], [10], [14]. To benchmark these metrics, synthetic networks such as those generated by the LFR benchmark are commonly used.

Classical Community Detection Algorithms

Classical community detection algorithms form the foundation of modern network analysis and have been extensively studied and applied across various domains [11]. These methods can be categorized based on their underlying principles: modularity optimization, random walk-based approaches, label propagation, spectral methods, and flow simulation. This section provides a comprehensive overview of the most prominent classical algorithms and their characteristics.

Modularity-Based Algorithms

Louvain Algorithm

The Louvain method, introduced by Blondel et al., is one of the most widely adopted algorithms for community detection in large networks [24]. It operates through a greedy optimization approach that maximizes modularity, a quality function measuring the density of edges within communities compared to edges between communities.

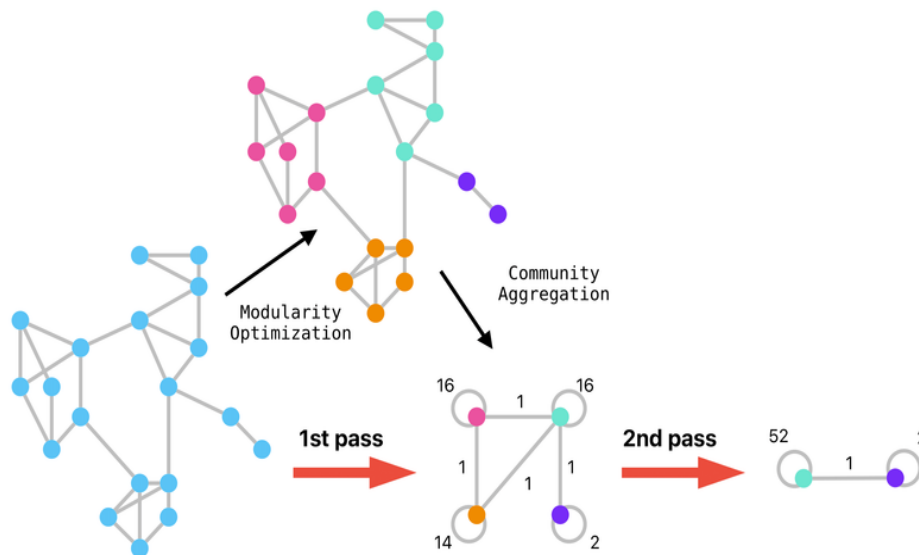


Figure 9 : Louvain Algorithm

The algorithm proceeds in two phases that are iterated recursively. In the first phase, each node is initially assigned to its own community. The algorithm then evaluates the modularity gain obtained by moving each node to neighboring communities, selecting the move that yields the highest positive gain. This process continues until no further improvement is possible. In the second phase, a new network is constructed where nodes represent the communities identified in the first phase, and edges between these super-nodes are weighted by the sum of edges between nodes in the corresponding communities [11].

The Louvain algorithm's primary strengths lie in its computational efficiency and scalability, making it suitable for networks with millions of nodes. It runs in near-linear time complexity and consistently produces high modularity values [26]. However, the algorithm suffers from significant limitations. Research has demonstrated that Louvain may yield arbitrarily badly connected communities, with up to 25% of communities being poorly connected and up to 16% being internally disconnected, especially when the algorithm is applied iteratively [24]. This occurs when nodes acting as bridges between groups are moved to new communities, leaving the original community fragmented into disjoint subgraphs.

Leiden Algorithm

The Leiden algorithm was specifically developed to address the fundamental deficiencies of the Louvain method. Introduced by Traag et al. [24] in 2019, it guarantees that all detected communities are well-connected and provides explicit mathematical guarantees on partition quality.

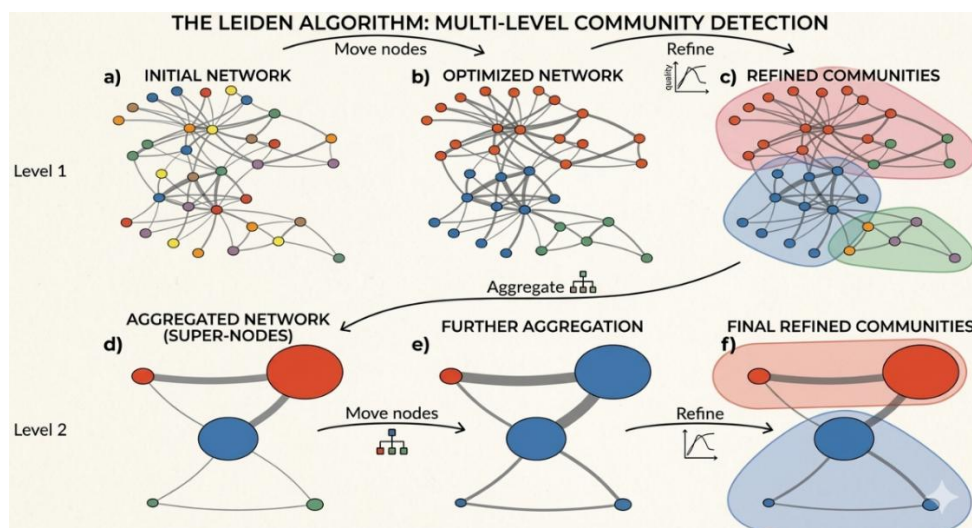


Figure 10 : Leiden Algorithm

The Leiden algorithm incorporates three key innovations. First, it introduces a refinement phase that subdivides communities and then moves nodes between these refined partitions, ensuring internal connectivity. Second, it uses a smart local move strategy that considers only nodes whose neighborhoods have changed, reducing unnecessary computations. Third, when constructing the aggregate network for the next iteration, Leiden merges only nodes within well-connected subsets of communities rather than entire communities [24].

Comparative studies show that Leiden consistently outperforms Louvain across multiple metrics [25]. It achieves higher modularity values (typically 0.7453 versus 0.5942 for Louvain on real-world networks), demonstrates significantly shorter execution times (averaging 22.52 seconds compared to

Louvain's longer runtimes on Twitter networks), and provides more stable partitions with better-defined community boundaries [25]. The algorithm converges to a partition where all subsets of all communities are locally optimally assigned, a property that Louvain cannot guarantee [24]. In a case study analyzing Twitter communities during the 2022 war in Ukraine, Sliwa et al. [25] found that Leiden detected substantially more communities (97 compared to Louvain's 41) and produced partitions with higher average modularity scores.

Random Walk-Based Algorithms

Walktrap Algorithm

The Walktrap algorithm, proposed by Pons and Latapy, leverages the intuition that random walks on a graph tend to remain trapped within densely connected regions [27]. The algorithm defines distances between vertices based on random walk transition probabilities and uses hierarchical agglomerative clustering to merge communities.

The method begins by computing transition probabilities for random walkers of a specified length (typically 4 steps). The distance between two nodes is calculated based on the similarity of their random walk probability distributions to all other nodes in the network. Communities are then identified by iteratively merging the pair of communities that results in the smallest increase in within-community distances [11].

Walktrap demonstrates strong performance on networks with clear flow-based community structure [27]. Studies indicate it achieves relatively better accuracy and smaller standard deviations for larger networks ($N > 1000$) and maintains stable performance across varying network densities [11], [26]. The algorithm has time complexity of $O(n^2m)$ or $O(n^2 \log n)$ on sparse networks. It is particularly well-suited for applications involving transmission of information, communication, behaviors, or infectious disease spread, as it naturally captures flow dynamics [27]. However, Walktrap's hierarchical nature means it requires selection of an appropriate cut point in the dendrogram to obtain the final partition.

Infomap Algorithm

Infomap, developed by Rosvall and Bergstrom, approaches community detection from an information-theoretic perspective. The algorithm seeks to minimize the map equation, which represents the expected description length of a random walk on the network using a two-level coding scheme [27].

The key insight is that if a network has community structure, a random walker will spend long periods within communities before occasionally jumping between them. Infomap exploits this by assigning short code words to nodes within the same community (using community-specific codebooks) and longer codes for inter-community transitions. The optimal partition is the one that minimizes the total information needed to describe the random walker's trajectory [11].

Infomap consistently ranks among the top-performing algorithms in comparative studies [11], [26], [27]. For small networks ($N \leq 1000$), it achieves high accuracy with very small standard deviations, demonstrating excellent repeatability. The algorithm has linear time complexity $O(m)$ and scales well to large networks [26]. It shows a steep change in performance at a mixing parameter around $\mu = 0.6$, after which accuracy degrades [11]. Infomap is particularly effective for directed networks and networks with well-defined flow patterns. Studies comparing it to Walktrap show high overlap in detected communities (Adjusted Rand Index of 0.72), though Infomap typically requires fewer iterations to converge and does not require manual selection of partition granularity [26].

Markov Clustering (MCL)

The Markov Clustering algorithm simulates flow in a graph using two operators: expansion and inflation. The expansion operator computes random walks of higher length by raising the adjacency matrix to a power, allowing nodes to reach distant neighbors. The inflation operator strengthens strong connections and weakens weak ones by raising matrix elements to a power and renormalizing columns [28].

MCL iteratively applies expansion and inflation until the matrix converges to a form interpretable as disconnected components, which represent communities. Unlike many algorithms, MCL does not require pre-specification of the number of communities the partition emerges naturally from network topology [28].

However, MCL's performance heavily depends on the inflation parameter setting, making it sensitive to parameter selection. Studies show that while MCL performs well on synthetic networks and biological networks where it was originally developed, it generally produces results inferior to Louvain and Infomap on social networks [26], [28]. Recent work by Papadopoulos et al. [28] has demonstrated that pre-weighting the network using geometric or similarity measures significantly improves MCL's community detection performance. Their experiments show that geometrically inspired pre-weighting can increase the Normalized Mutual Information (NMI) of MCL from approximately 0.4 to 0.8 on certain benchmark networks, bringing it closer to current state-of-the-art methods [28].

Spinglass Algorithm

The Spinglass algorithm, inspired by statistical physics, formulates community detection as finding the ground state of a Potts spin glass model. Each node is assigned to a spin value, and the algorithm minimizes an energy function that favors configurations where connected nodes have the same spin [11].

The method uses simulated annealing to explore the configuration space and find low-energy states corresponding to good community partitions. For small networks ($N \leq 1000$), Spinglass achieves high accuracy comparable to Infomap and Walktrap [11], [26]. However, its computational complexity is prohibitively high for large networks, with running times that make it impractical for networks beyond a few thousand nodes. The algorithm is typically limited to networks smaller than 10,000 nodes due to computational constraints [26].

Divisive Algorithms

Girvan–Newman (GN) Algorithm

The Girvan-Newman algorithm is one of the earliest and most influential divisive methods for community detection [11]. It operates by iteratively identifying and removing edges with high betweenness centrality edges that lie on many shortest paths between node pairs.

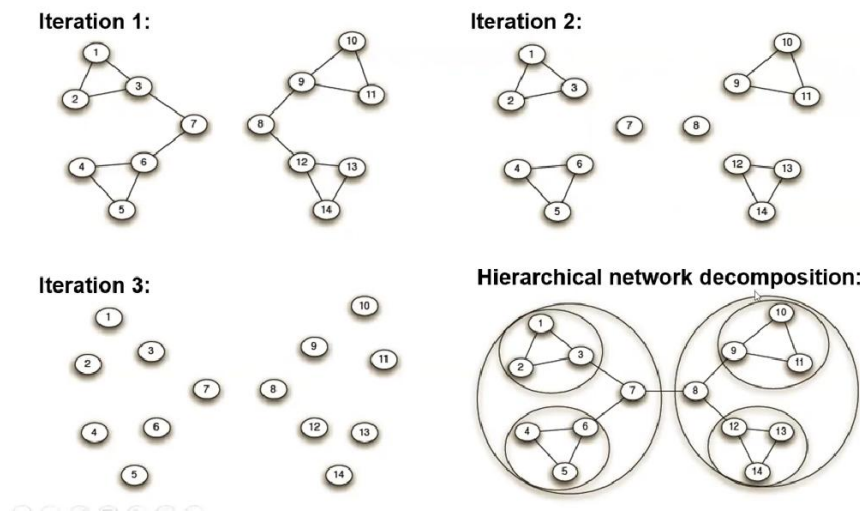


Figure 11 : Girvan Newman Algorithm

The algorithm begins with the complete network and repeatedly calculates edge betweenness for all edges, removes the edge with highest betweenness, and recalculates betweenness values. This process continues until all edges are removed or a desired number of communities is reached. A dendrogram captures the hierarchical community structure, and the partition with maximum modularity is typically selected [11].

Despite its historical importance and conceptual elegance, the GN algorithm has severe computational limitations. Its time complexity is $O(m^2n)$ or $O(n^3)$ on sparse graphs, making it impractical for networks larger than a few thousand nodes [26]. Studies show it cannot complete execution on networks with 20,000+ nodes even after multiple days of computation [11]. Additionally, the algorithm's performance degrades when multiple edges have identical betweenness values, creating ambiguity in edge selection. For these reasons, GN has largely been superseded by more efficient methods, though it remains valuable for small networks and as a conceptual baseline [27].

Kernighan-Lin Algorithm

The Kernighan-Lin algorithm is one of the earliest heuristic methods for graph partitioning, introduced by Kernighan and Lin in 1970 [43]. Unlike modern modularity optimization methods, this algorithm was designed to partition a graph into two equal-sized subsets while minimizing the number of edges crossing between them.

The algorithm works through an iterative greedy strategy. It begins with an arbitrary initial partition of the nodes into two groups of specified sizes. At each step, the algorithm evaluates all possible swaps of node pairs between the two groups, calculating the gain in cut size for each swap. The pair that produces the largest reduction is swapped, even if the gain is negative, allowing the algorithm to escape local optima. This process continues until no further improvement is possible [43].

The primary advantage of the Kernighan-Lin algorithm is its ability to find high-quality partitions through its sophisticated local search strategy. However, it has several limitations. The algorithm requires the number of communities to be specified in advance and is typically designed for bisection rather than arbitrary community numbers. It also has relatively high computational complexity, making it impractical for very large networks. Despite these limitations, the Kernighan-Lin algorithm remains influential as one of the foundational methods in graph partitioning and community detection [43].

Several extensions have been proposed to address its limitations. The Combo algorithm combines merging, splitting, and node-moving strategies to optimize modularity across different objective functions. The Modified Fine-Tuning Strategy adds an additional refinement step to the recursive bisection process, allowing nodes to move across existing communities after the standard bisection, which improves accuracy without increasing computational complexity [43].

Label Propagation Methods

Label Propagation Algorithm (LPA)

The Label Propagation Algorithm is a near-linear time method based on local information diffusion [27]. Each node is initially assigned a unique label. In each iteration, nodes adopt the label that is most frequent among their neighbors, with ties broken randomly or using node degree as a tiebreaker.

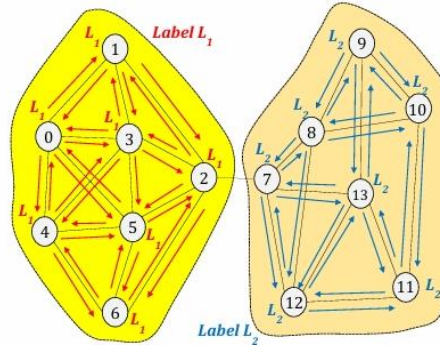


Figure 12 : LPA Algorithm

The process continues until labels stabilize, meaning no node changes its label. Nodes sharing the same final label belong to the same community. LPA's primary advantage is its simplicity and speed it runs in $O(m)$ time and requires no parameter tuning or prior knowledge of community number [11], [26].

However, LPA suffers from significant stability issues. Different runs on the same network can produce dramatically different results due to its stochastic nature [11], [26]. Studies report that LPA has much larger standard deviations than other algorithms, with outputs that are fundamentally unstable. The algorithm frequently converges to a single giant community rather than meaningful partitions, particularly in networks lacking strong community structure [26]. While some nodes quickly reach consensus, others may oscillate between labels indefinitely. Gates et al. [27] note that LPA's lack of stability makes it problematic for applications requiring reproducible results. Despite these limitations, LPA achieves competitive performance on certain benchmark networks, particularly when combined with memory mechanisms or other enhancements [11].

Fast Label Propagation Algorithm

FLPA introduces optimizations to the standard LPA to improve convergence speed and stability. It employs techniques such as asynchronous updates (where nodes update labels immediately rather than synchronously), preferential treatment of high-degree nodes in the update order, and strategic initialization of labels based on network topology [26].

Some variants incorporate a memory mechanism (MemLPA) where nodes remember past label states and use this historical information in decision-making. Experiments show that MemLPA outperforms

standard LPA for mixing parameters $\mu \in [0.5, 0.7]$ and performs comparably to Infomap and Walktrap in this range [11]. For $\mu \geq 0.7$, MemLPA demonstrates superior performance compared to most algorithms except edge betweenness methods.

However, even with these improvements, FLPA variants still exhibit some instability, though less severe than standard LPA [26]. The memory-based approaches tend to find a greater number of smaller communities, which can affect partition quality metrics depending on the application requirements.

Label Propagation Algorithm with Modularity and Node Importance (LPA-MNI)

The standard Label Propagation Algorithm suffers from randomness and instability, as different runs can produce dramatically different community partitions. LPA-MNI addresses these issues by integrating modularity optimization and node importance into the label propagation framework [43].

The algorithm operates in two main stages. First, it identifies initial communities through modularity maximization, assigning unique labels to nodes within these initial communities. This initialization step significantly reduces the method's susceptibility to randomness. Second, node importance, determined by degree centrality, dictates the order of label updates. Nodes with higher importance are updated first, ensuring a systematic and stable propagation process [43].

Extensive testing on real-world and synthetic networks demonstrates that LPA-MNI achieves enhanced accuracy, robustness, and efficiency compared to the standard LPA. The method produces higher modularity values and more consistent community numbers, confirming its effectiveness. The introduction of a deterministic and accurate label update mechanism significantly improves upon the traditional label propagation method's stability, making it a more reliable choice for practical applications [43].

Constrained Label Propagation Algorithm (CLPA)

CLPA reformulates the label propagation algorithm as an optimization problem to address its inherent drawbacks. The key innovation is the introduction of a modified objective function that incorporates constraints to penalize less desirable solutions, such as the trivial solution where all vertices share the same label [43].

The algorithm proposes several constraint variants tailored to different network types. A modularity-maximizing variant focuses on optimizing community quality, while specialized adaptations exist for bipartite networks. Through comparative analysis of both synthetic and real-world networks, CLPA demonstrates its ability to evade the pitfalls of standard LPA while showing versatility in handling different network structures [43].

This methodology enables the detection of more meaningful communities by effectively leveraging constraints within the optimization process. CLPA offers a nuanced and flexible tool for network analysis, providing researchers with greater control over the community detection process through its constraint-based formulation [43].

FluidC Algorithm

FluidC (Fluid Communities) is a propagation-based algorithm that treats communities as fluids expanding through the network. Each community is initialized at a random seed node and grows by incorporating neighboring nodes based on density and flow dynamics [26].

The algorithm simulates the process of fluids flowing and displacing each other, where denser regions (communities) naturally emerge. Like LPA, FluidC is computationally efficient with complexity near-linear time. It performs particularly well on large-scale networks; studies show it achieves high and stable performance (ARI and Purity around 0.9) on LFR benchmark networks with up to 500,000 nodes [26].

However, FluidC requires the number of communities as an input parameter, which is often unknown in real-world scenarios [26], [27]. Additionally, the algorithm produces different results across runs due to its stochastic initialization and propagation process, though this instability is less pronounced than in standard LPA. On some networks, FluidC's ARI values drop nearly to zero, indicating highly inconsistent partitions depending on initialization [26].

Spectral Methods

Eigenvector-Based Method

Spectral methods leverage the eigenvectors and eigenvalues of matrices derived from the graph, such as the adjacency matrix, Laplacian matrix, or modularity matrix [11]. Newman's leading eigenvector method is a prominent example that uses the eigenvector corresponding to the largest positive eigenvalue of the modularity matrix to recursively bisect the network.

The algorithm computes the modularity matrix B , where $B_{\{ij\}} = A_{\{ij\}} - \frac{(k_i k_j)}{2m}$ with A being the adjacency matrix, k representing node degrees, and m the total number of edges. The leading eigenvector is calculated, and nodes are assigned to communities based on the sign of their corresponding eigenvector components. This process is applied recursively to each subcommunity until modularity cannot be further improved [11].

Spectral methods have theoretical guarantees under certain network models (such as the stochastic block model) and can detect communities with clear structural separation [27]. However, comparative studies show that the leading eigenvector method tends to overshoot results and perform worse than optimization-based methods like Louvain [11], [26]. The method has time complexity of $O(n(m+n))$ or $O(n^2)$ on sparse networks.

Recent advances have combined spectral methods with other techniques. For instance, using node embeddings (like Node2Vec) followed by spectral clustering shows improved performance, particularly when enhanced with neural network-based embeddings for networks with high clustering coefficients [26]. Spectral approaches also face challenges in determining the appropriate number of communities and can struggle with networks containing overlapping or hierarchical community structures [27].

Seed-centric Approaches

SimCMR (Similarity-Based Community Merge/Refinement)

SimCMR represents a class of algorithms that combine multiple strategies for community detection [26]. These methods typically begin with an initial partition (often from a fast algorithm) and then refine it by merging or splitting communities based on similarity measures.

The approach involves computing node or community similarities using various metrics (structural equivalence, random walk probabilities, or embedding distances) and iteratively improving the partition by merging similar communities or refining boundary nodes. Some variants use a two-stage process: first detecting candidate communities quickly, then refining boundaries using more sophisticated similarity computations [26].

SimCMR-type approaches can incorporate both global and local information, balancing accuracy with computational efficiency. They often achieve better stability than pure propagation methods while maintaining reasonable runtime. However, the performance depends heavily on the choice of similarity metric and the initial partition quality [26].

Modern Approaches

While classical algorithms have been the foundation of community detection for many years, modern approaches have emerged to overcome their limitations in terms of scalability, expressiveness, and the ability to leverage node attributes. These methods fall into two major categories: graph embedding techniques, which learn low-dimensional vector representations of nodes, and deep learning approaches, which use neural network architectures specifically designed for graph-structured data.

Graph Embeddings

Graph embedding methods aim to represent each node of a network as a dense, low-dimensional vector in a continuous space such that structurally similar nodes are mapped close together. These representations can then be used with standard machine learning techniques, such as k-means clustering, to perform community detection [7], [8].

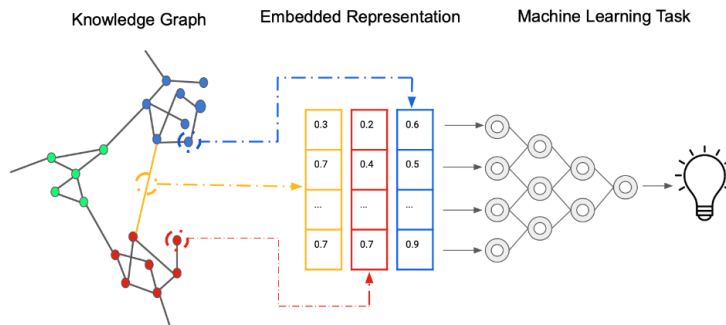


Figure 13 : Illustration of the graph embedding pipeline from knowledge graph to feature vectors.

DeepWalk

DeepWalk was one of the first methods to apply natural language processing techniques to graph representation learning. Proposed by Perozzi et al. in 2014, it draws an analogy between sentences in a corpus and sequences of nodes visited during random walks on a graph [29].

The algorithm works in two steps. First, it generates many truncated random walks starting from each node in the graph. These walks produce sequences of nodes that reflect the local neighborhood structure. Second, these sequences are fed into the Word2Vec Skip-gram model, which learns node embeddings by predicting the context nodes surrounding each node within a fixed window [29].

The key insight behind DeepWalk is that nodes which frequently co-occur in random walks tend to be structurally related, often belonging to the same community. As a result, their embeddings end up close together in the vector space. Once embeddings are learned, standard clustering algorithms such as k-means can be applied to group nodes into communities [29].

DeepWalk scales well to large networks and runs efficiently in an online manner, making it suitable for dynamic graphs. However, its random walks are entirely uniform, meaning it does not distinguish between exploration of different structural properties. Additionally, DeepWalk is a transductive method, which means it cannot directly generate embeddings for unseen nodes without retraining [8].

Node2Vec

Node2Vec, introduced by Grover and Leskovec, extends DeepWalk by introducing a more flexible and controllable random walk strategy. The main contribution is a biased second-order random walk mechanism governed by two parameters: the return parameter p and the in-out parameter q [29].

The parameter p controls the likelihood of immediately revisiting a node already visited, effectively regulating how much the walk explores or returns to its starting area. The parameter q balances the walk between two different strategies: Breadth-First Sampling (BFS), which explores the immediate neighborhood and captures homophily, and Depth-First Sampling (DFS), which moves further away from the starting node and captures structural equivalence. By tuning these two parameters, Node2Vec can produce embeddings that reflect either community membership (when p is high and q is low) or structural roles within the network (when p is low and q is high) [29].

Comparative studies show that Node2Vec consistently outperforms DeepWalk on community detection tasks, particularly on networks where both community membership and structural roles are important. However, like DeepWalk, Node2Vec is transductive and does not generalize to unseen nodes. Its performance also depends on careful tuning of p and q , which may require domain knowledge or cross-validation [8].

GraphSAGE

GraphSAGE (Graph SAmple and aggreGatE) addresses a key limitation of DeepWalk and Node2Vec by introducing an inductive learning framework. Proposed by Hamilton et al., it learns an aggregation function over node neighborhoods rather than learning fixed embeddings for each node [30].

At inference time, the embedding of a node is computed by sampling a fixed number of neighbors and applying a learned aggregation function (such as mean pooling, max pooling, or an LSTM) to combine the features of those neighbors. This process is applied over multiple hops, allowing information from further parts of the graph to be incorporated. Because the aggregation function is shared across all nodes, GraphSAGE can generate embeddings for nodes that were not present during training, making it inherently inductive [30].

For community detection, the embeddings produced by GraphSAGE can be clustered using standard algorithms. Its ability to incorporate both structural information and node features (such as user profile attributes) makes it especially useful in attributed social networks. Hamilton et al. demonstrated that GraphSAGE significantly outperforms DeepWalk and Node2Vec on node classification and link prediction tasks in large-scale graphs such as Reddit and Pinterest [30].

Deep Learning on Graphs

Deep learning approaches for graphs extend classical neural network architectures to handle the irregular, non-Euclidean structure of graph data. Unlike images or text sequences, graphs have no fixed spatial locality or ordering, which makes conventional convolutional or recurrent networks unsuitable. A family of models known as Graph Neural Networks (GNNs) addresses this challenge by defining learnable operations that propagate and aggregate information along graph edges [15].

Graph Neural Networks (GNN)

Graph Neural Networks (GNNs) constitute a broad class of models that learn representations for nodes, edges, or entire graphs by iteratively aggregating information from local neighborhoods. The core idea, shared across most GNN variants, is a message-passing mechanism: at each layer, every node collects feature vectors from its neighbors, aggregates them (typically through summation or averaging), and updates its own representation [15].

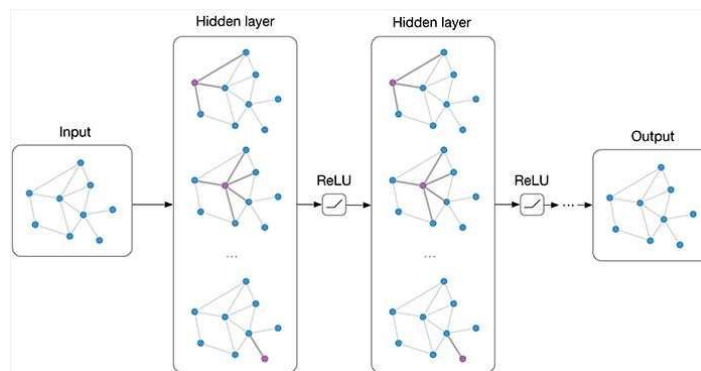


Figure 14 : Architecture of a Multi-layer Graph Neural Network.

After k layers, the representation of a node encodes information from its k -hop neighborhood. This multi-hop aggregation makes GNNs naturally suited for community detection, as nodes belonging to the same community tend to have similar neighborhood structures. GNNs can be trained in a supervised manner using labeled node data, in a semi-supervised manner with partial labels, or in an unsupervised manner using reconstruction-based objectives [15].

Zhou et al. [15] provide a comprehensive review of GNN architectures and applications, noting that GNNs have achieved state-of-the-art results on node classification, link prediction, and graph classification benchmarks, making them a powerful backbone for downstream community detection tasks.

Graph Convolutional Networks (GCN)

Graph Convolutional Networks (GCNs), introduced by Thomas Kipf and Max Welling, are among the most influential and widely used variants of Graph Neural Networks (GNNs). They extend the concept of convolution from regular grid-structured data, such as images, to irregular graph structures by relying on principles from spectral graph theory [30].

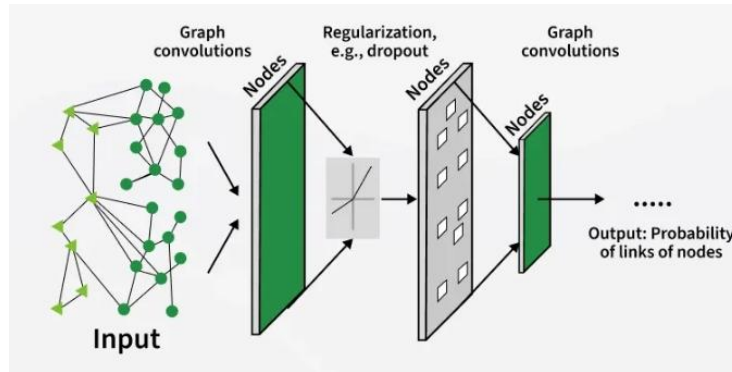


Figure 15 : Schematic representation of GCN layers.

In a GCN layer, each node updates its representation by combining its own features with those of its neighboring nodes. This aggregation is performed in a normalized manner to ensure stability and balanced contributions from different nodes, followed by a non-linear transformation. By stacking multiple layers, the model can capture information from progressively larger neighborhoods, enabling it to learn both local and global structural patterns in the graph [30].

For community detection tasks, GCNs are often employed within an encoder–decoder framework. The encoder uses graph convolution operations to transform nodes into a latent embedding space, while the decoder leverages these embeddings to reconstruct the graph structure or to infer community memberships. This approach has shown strong performance on benchmark datasets, often producing more coherent and semantically meaningful communities compared to traditional methods, especially when node features are available [15], [30].

Despite their effectiveness, GCNs present several challenges in practice. One major issue is over-smoothing, where node representations become increasingly similar as more layers are added, reducing their discriminative power. Additionally, scaling GCNs to large graphs can be computationally demanding. To address these limitations, recent research has introduced techniques such as neighbor sampling, mini-batch training, and graph partitioning, which improve both efficiency and scalability [15].

Graph Attention Networks (GAT)

Standard Graph Convolutional Networks treat all neighbors equally when updating a node's representation. Graph Attention Networks improve upon this by learning to assign different levels of importance to different neighbors, as surveyed by Su et al. [42].

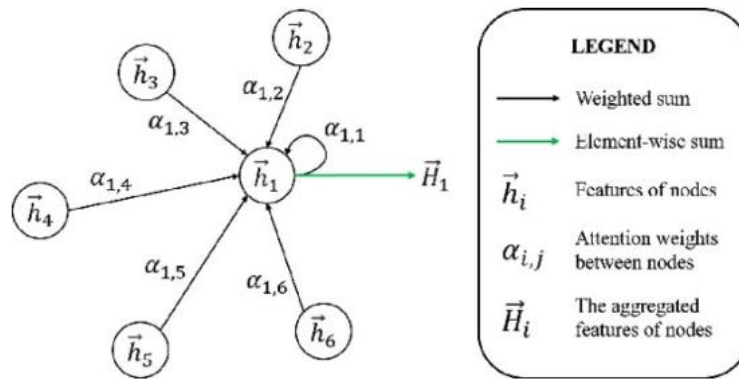


Figure 16 : Graph Attention Network

The key idea is that not all connections in a social network matter the same way. A close friend should have more influence on a user's community membership than a casual acquaintance. GAT automatically learns these importance weights through an attention mechanism. For each node, the model calculates how much attention to pay to each neighbor, then aggregates neighbor information accordingly.

For community detection, GAT offers several practical advantages. First, it can filter out noisy or irrelevant connections that might otherwise mislead the detection process. Second, it handles networks with multiple relationship types, such as friendship, family, and professional connections on the same platform. Third, the learned attention weights can be interpreted, revealing which types of relationships are most predictive of community belonging [42].

Several researchers have successfully applied GAT to community detection. DMGI uses attention to combine information from different types of relationships in multiplex networks. MAGNN applies attention along metapaths in heterogeneous networks, capturing complex semantic relationships. HDMI extends this approach with semantic attention to fuse embeddings across multiple network layers [42].

Autoencoder-Based Methods

Autoencoders are a popular deep learning architecture for unsupervised community detection. They learn compact node representations by compressing and reconstructing a network's adjacency matrix through an encoder-decoder structure with a narrow bottleneck. Nodes that share similar connectivity patterns end up with similar compressed representations, enabling standard clustering algorithms like k means to effectively detect communities [42].

Several important variants exist. Stacked autoencoders use multiple hidden layers to learn hierarchical features. Denoising autoencoders are trained on corrupted versions of the input, making them robust to missing or noisy edges. Variational autoencoders learn probability distributions over embeddings rather than fixed points, capturing uncertainty in community assignments [42].

Specific methods include Semi DRN, which incorporates modularity optimization into autoencoder training, DANE, which uses two parallel autoencoders for network structure and node attributes, and MGAE, which marginalizes over multiple corrupted inputs for greater robustness. Autoencoders are particularly well suited for social network community detection because they require no labeled training data, scale efficiently to large networks, and can easily incorporate multiple information sources such as user profiles and interaction histories alongside network structure [42].

Variational Graph Autoencoders

While the previous section discussed deterministic autoencoders for community detection, the Variational Graph Autoencoder (VGAE), introduced by Kipf and Welling, extends this framework with a probabilistic approach [4]. Unlike standard autoencoders that learn fixed embeddings, VGAE learns a probability distribution over the latent space, capturing uncertainty in node representations and community assignments [1].

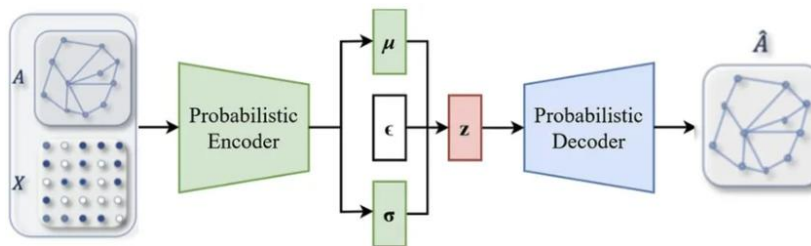


Figure 17: Architecture of a VGAE with encoder and decoder

The architecture follows the same encoder-decoder principle as deterministic autoencoders, but with a key difference: the encoder outputs parameters of a Gaussian distribution (mean and variance vectors) rather than fixed embeddings. Training optimizes a variational lower bound that balances reconstruction accuracy against a KL divergence regularizer, which prevents overfitting and promotes smooth latent spaces [4].

For community detection, VGAE offers advantages over deterministic autoencoders. The probabilistic representation naturally handles ambiguous community boundaries, where nodes at the periphery of communities exhibit higher variance. Choong et al. demonstrated that VGAE with dual optimization achieves competitive community detection performance with faster convergence compared to standard autoencoders [1]. Murata's research further validated that encoding graph structure using multiple Gaussian distributions enables effective unsupervised community extraction [3].

The main strength of VGAE lies in its robustness to noise and missing edges, making it particularly suitable for real-world social networks where interaction data is often incomplete [2].

Applications of Community Detection

Community Detection for Targeted Marketing and E-Commerce

Recent research demonstrates the practical value of community detection in marketing contexts. S. Paul *et al.* [17] introduced the Commerce Graph which is a dynamic network that links buyers, sellers, products, and transactions demonstrating that community detection using the Louvain algorithm enables effective customer segmentation. The study found that customers with higher centrality scores (degree, betweenness, closeness) play pivotal roles in the network, providing actionable insights for personalized promotions and optimized business operations.

In the domain of customer segmentation, Nasirzonouzi [21] proposed a bipartite graph model of customers and products using the "Online Retail II" dataset.

By converting the bipartite graph into a customer-to-customer network and applying community detection, the study revealed complex interrelations that traditional methods (K-means clustering) fail to capture effectively. Centrality metrics were used to quantify customer influence, enabling highly targeted marketing strategies based on network roles such as key influencers and bridging nodes.

A. Kanavos *et al.* [19] applied Louvain and Label Propagation algorithms to Twitter datasets to delineate community structures and identify influential users. Their findings demonstrate that community structures significantly influence information dissemination and user engagement, offering strategic insights for influencer engagement across marketing, politics, and public health sectors.

Community Detection for Recommender Systems

Community detection has emerged as a powerful approach for addressing fundamental challenges in recommender systems. Sánchez-Moreno et al. [18] addressed the cold-start problem (new users with no interaction history) and the gray-sheep problem (users with unusual preferences) in music recommender systems. Their proposed user similarity metric based on social network structure and user ratings captures homophily effects in implicit communities, significantly improving recommendation reliability for users with limited or atypical preference data.

Saikalyan et al. [22] proposed a recommender system using local community detection via personalized PageRank and the Bethe Hessian matrix to estimate community numbers, followed by non-negative matrix factorization for overlapping community detection. The approach defines a belongingness factor for each user-community relationship and effectively addresses the cold-start problem on MovieLens 1M and Facebook Ego Network datasets.

For GNN-based recommender systems, Hu et al. [23] introduced the General Graph Coarsening Recommendation Framework (GGCRF), which uses community detection to divide user-item graphs into smaller segments, forming super-nodes that encapsulate groups of similar nodes. This framework overcomes the limitation of GNNs that primarily focus on closely located nodes, addressing the long-tail effect prevalent in recommendation systems.

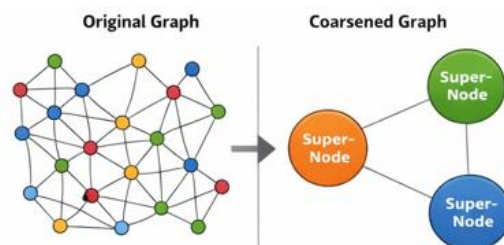


Figure 18 : Original vs Coarsened Graphs

Integrating Textual Semantics

S. K. S. et al. [20] proposed a novel framework combining pre-trained language models (PLMs) and graph neural networks (GNNs) for community detection in text-attributed graphs. Using datasets from Goodreads (books, reviews) and Amazon (movies), the study demonstrates that integrating textual embeddings with graph structure significantly improves community detection accuracy for recommendation systems. This approach bridges the gap between semantic content and topological structure, enabling more nuanced community identification in text-rich domains.

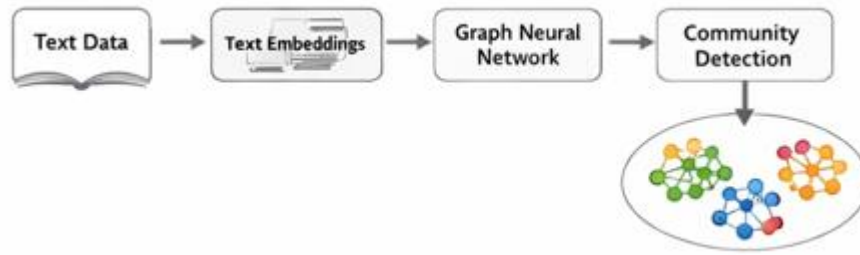


Figure 19 : Semantic-topological community detection framework

Standard Benchmark Datasets for Community Detection

To fairly compare different community detection methods, researchers need standardized test networks. Synthetic benchmarks such as LFR offer controlled conditions for experimentation, but real-world networks remain essential for assessing how algorithms perform on authentic data. Over the past several decades, a collection of relatively small networks has become widely adopted as the standard test suite in this field. These datasets are popular because they are openly accessible, possess known or interpretable group structures, and span different types of real systems [11], [26].

Zachary's Karate Club

The Karate Club network stands as the most widely recognized dataset in social network analysis. Wayne Zachary collected this data in 1977, mapping friendships among 34 members of a university karate club [38]. What makes this network particularly valuable is its natural ground truth: an actual conflict caused the club to divide into two separate factions following a disagreement between the instructor and the president. This real-world split provides an ideal reference for testing community detection algorithms. The network consists of 34 nodes connected by 78 edges. Its modest size facilitates easy visualization and rapid algorithm testing, yet its structure is sufficiently nuanced to highlight performance differences between methods [38].

According to published surveys, the Karate Club dataset appears in over 26% of community detection studies, making it the most frequently used benchmark in the literature [38], [39].

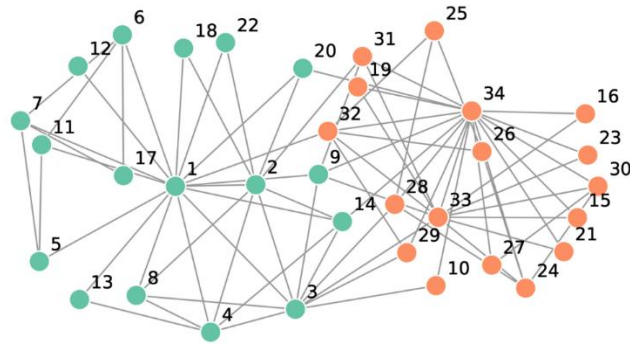


Figure 20 : Zachary's Karate Club

Dolphin Social Network

David Lusseau compiled the Dolphin network by observing a population of 62 bottlenose dolphins in Doubtful Sound, New Zealand [3]. Between 1995 and 2001, researchers identified each adult dolphin through natural markings on their dorsal fins and recorded which individuals were frequently seen together. An edge between two dolphins indicates that they were observed in close association more often than would be expected randomly. The network naturally separates into two primary groups, mirroring the actual social division observed in the fjord.

With 62 nodes and 159 edges, this dataset offers greater complexity than the Karate Club, including instances where some individuals may belong to multiple groups. When properly partitioned, the network yields a modularity score of approximately 0.5258, making it a useful test case for evaluating algorithmic stability and detection accuracy [26], [38].

American College Football Network

Michelle Girvan and Mark Newman constructed the Football network from the schedule of games played during the 2000 American college football regular season [26]. In this network, 115 nodes represent individual teams, and 613 edges represent games played between them. The ground-truth communities correspond to the 12 official conferences to which teams belong. Because teams compete more frequently against opponents within their own conference than against those from other conferences, the network exhibits a clear community structure.

This dataset is especially valuable because the true community labels are known in advance, enabling precise quantitative evaluation of detection performance. Under optimal partitioning, the Football network achieves a modularity of approximately 0.6006 [38].

Political Books Network (Polbooks)

V. Krebs created the Polbooks network by analyzing co-purchase relationships among books about American politics sold on Amazon around the time of the 2004 presidential election [40]. In this network, each node represents a book, and edges indicate that the same buyers frequently purchased both books together. The network displays a clear political division, with books falling naturally into three clusters corresponding to liberal, conservative, and neutral political leanings. What makes this dataset particularly interesting is that the ground-truth communities derive from semantic content (political alignment) rather than purely topological features. This allows researchers to test whether algorithms can recover meaningful semantic groupings based solely on network structure [40]. The network contains 105 nodes and 441 edges, with an average degree of 8.4 [40].

Primary School Contact Network (Thiers)

Stehlé and colleagues collected the Primary School network in 2009 by recording face-to-face contacts among 242 children and teachers at a school in Lyon, France [41]. The research team equipped participants with radio-frequency identification (RFID) devices that measured physical proximity at 20-second intervals over two consecutive days. The resulting network captures who interacted with whom and for how long, with edge weights representing contact duration. This dataset stands out for its high temporal resolution and for representing physical social interactions rather than virtual ones.

With 242 nodes and approximately 125,000 weighted edges, it ranks among the larger benchmark datasets commonly used in literature. The ground-truth communities correspond to school classes, since children naturally interact most frequently with classmates [41].

Facebook Social Network

The Facebook dataset, introduced by McAuley and Leskovec [44], consists of anonymized ego networks collected from survey participants. In network analysis, an ego-network is a subgraph centered around a single user (the "ego"), including their immediate connections (friends) and the connections among those friends. The dataset contains 4,039 nodes and 88,234 edges, with ground truth communities corresponding to user-created "circles" such as family, classmates, or coworkers [44]. This dataset is valuable for evaluating algorithms on realistic social structures with overlapping communities, where a single user naturally belongs to multiple distinct social groups.

LastFM Social Network

The LastFM dataset, also collected by McAuley and Leskovec [44], captures social connections and musical listening habits on the LastFM online music platform. It contains 28,281 nodes (users) and 92,752 edges (friendship links), with ground-truth communities derived from user membership in interest groups corresponding to artists, genres, or listening clubs [44]. This dataset combines social structure with explicit behavioral affinities, making it suitable for evaluating homophily driven community formation in medium-scale social graphs.

Additional Datasets (VS13 and VS15)

The VS13 and VS15 datasets, occasionally referred to as the "versus" datasets in published work, appear in certain comparative studies of community detection methods [26]. Researchers typically employ these networks when evaluating algorithm performance across graphs with differing structural characteristics or when comparing modularity-based approaches against alternative techniques. Although less frequently used than the Karate Club or Football networks, VS13 and VS15 offer additional test cases for assessing algorithmic robustness under various network topologies [26].

LFR Benchmark

The Lancichinetti Fortunato Radicchi (LFR) benchmark is the most widely used standard for evaluating community detection algorithms on synthetic networks. Proposed to address the limitations of the Girvan Newman (GN) benchmark such as homogeneous degree distributions and equal-sized communities it generates more realistic graph structures [31].

LFR produces networks with properties closer to real-world systems by drawing node degrees and community sizes from power-law distributions. This results in heterogeneous structures with hubs and communities of varying sizes, better reflecting social and information networks [11], [31].

A key parameter in LFR is the mixing parameter μ , which represents the fraction of edges a node shares with nodes outside its community. Lower values of μ indicate well-defined communities, while higher values make detection more challenging. When $\mu > 0.5$, communities in the strong sense no longer exist.

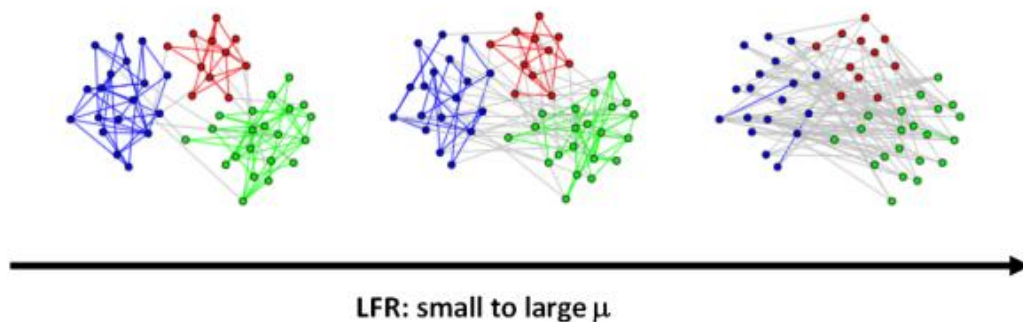


Figure 21 : Impact of the parameter μ on the detectability of clusters in heterogeneous networks.

In addition to μ , LFR includes parameters such as the number of nodes, degree distribution, and community size ranges, allowing the generation of diverse network configurations for systematic evaluation [11], [31].

Compared to GN, LFR is more challenging and better reveals algorithm limitations, which is why it is widely used in benchmarking studies. For example, Anuar et al. [26] evaluated multiple algorithms on LFR networks using metrics like NMI, ARI, and Purity, showing that performance varies significantly with μ .

LFR has been extended to support directed, weighted, and overlapping communities [31]. Despite its effectiveness, it has limitations, including scalability issues and unrealistic structures at extreme μ values [26]. Nonetheless, it remains the dominant benchmark in the field.

Foundations of Recommender Systems

Definition and Importance

Recommender systems are information filtering tools that predict the preferences or ratings a user would assign to items such as movies, products, songs, or news articles. These systems have become indispensable in the modern digital economy, helping users navigate the overwhelming abundance of choices available on platforms such as Netflix, Amazon, Spotify, and YouTube [35]. By personalizing content delivery, recommender systems improve user satisfaction, increase engagement, and drive business revenue. The core function of any recommender system is to take a user's known preferences and past behaviors as input, then generate a ranked list of items that the user is most likely to find appealing [36].

Classical Paradigms

Three main paradigms have historically dominated recommender system research and practice [35], [36].

Content-Based Filtering recommends items similar to those a user has liked in the past. This approach builds a profile for each user based on the features of items they have interacted with. For example, if a user frequently watches action movies starring a particular actor, the system will recommend other action movies featuring that same actor. The primary advantage of content-based filtering is its ability to recommend new items that have no prior interactions. However, it suffers from overspecialization, where recommendations become too narrow and fail to discover unexpected items that the user might still enjoy [35].

Collaborative Filtering is the most widely adopted paradigm. Instead of relying on item features, it leverages the "wisdom of the crowd": users who agreed in the past are likely to agree again in the future. Collaborative filtering comes in two main flavors. User-based collaborative filtering finds users with similar taste patterns to the target user and recommends items those similar users have liked. Item-based collaborative filtering finds items that are frequently liked together and recommends them to users who have shown interest in similar items. The primary strength of collaborative filtering is its ability to make recommendations in domains where item features are complex or unavailable. Its main weakness is the cold-start problem: new users with no interaction history cannot be effectively matched to similar users, and new items cannot be recommended until someone interacts with them [36], [37].

Hybrid Approaches combine content-based and collaborative filtering to overcome their respective limitations. A hybrid system might use content-based profiles to address the cold-start problem for new

items while relying on collaborative filtering for established items. Many commercial systems, including Netflix and Amazon, employ hybrid architectures to achieve robust performance across different scenarios [37].

Community Detection for Recommender Systems

Collaborative Filtering Enhanced by Community Structure

Sheng et al. [32] addressed the weakness of GNN-based collaborative filtering under sparse data conditions. Their method constructs a user–user network from historical user–item interaction data, then applies community detection to partition users into groups with shared interests. The discovered community memberships are used to mine users' latent attribute preferences, which are then injected into a GNN-based collaborative filtering model to produce final recommendations. The approach explicitly combines the local cohesion of communities with the high-order propagation capability of graph neural networks. Evaluated on two large real-world datasets, the proposed algorithm outperforms both standard collaborative filtering and state-of-the-art GNN-based recommendation methods, confirming that community-aware neighborhoods construction substantially reduces the impact of data sparsity on recommendation quality.

Tokala et al. [33] proposed the Community-Based Matrix Factorization (CBMF) framework, which directly integrates community detection into the matrix factorization pipeline to reduce computation time while improving recommendation quality.

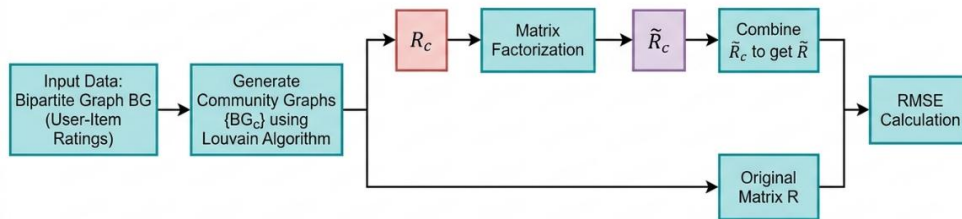


Figure 22 : Overview of the CBMF recommendation pipeline.

Rather than applying matrix factorization over the entire user-item rating network, the method first identifies communities within the rating graph using community detection algorithms, then applies matrix factorization locally within each community. This decomposition reduces the size of the matrices that need to be factorized, significantly lowering computational cost without sacrificing accuracy. The authors also incorporated the FANMF variant of non-negative matrix factorization, which accounts for user and item biases. Experiments show consistent improvements in RMSE over standard matrix factorization baselines, demonstrating that community structure in rating networks constitutes exploitable information for more efficient and accurate recommendations.

Addressing the Cold-Start Problem with Community Detection

Sánchez-Moreno et al. [18] tackled two well-known weaknesses in music recommender systems: the cold-start problem affecting new users with no interaction history, and the gray-sheep problem affecting users with unusual preferences. Their approach builds an implicit social network from user rating patterns and detects communities within it using structural similarity. A user similarity metric is then computed from both community membership and available ratings, capturing homophily effects between users who belong to the same community. This community-informed similarity significantly improves recommendation reliability precisely in cases where traditional collaborative filtering breaks down for new users and atypical users since community membership provides an alternative signal when rating data is scarce.

Combining Semantic Content and Community Structure

Rostami et al. [34] developed a hybrid recommender system for food recommendations that uses graph clustering to capture community structure among users and incorporates temporal dynamics into the similarity computation. Users are grouped via a weighted graph where edge weights reflect a time-aware similarity metric, giving more recent interactions higher influence. Community membership from this graph constrains the neighbourhood used during rating prediction, ensuring that recommendations are driven by users who are both structurally close in the network and behaviourally aligned over time.

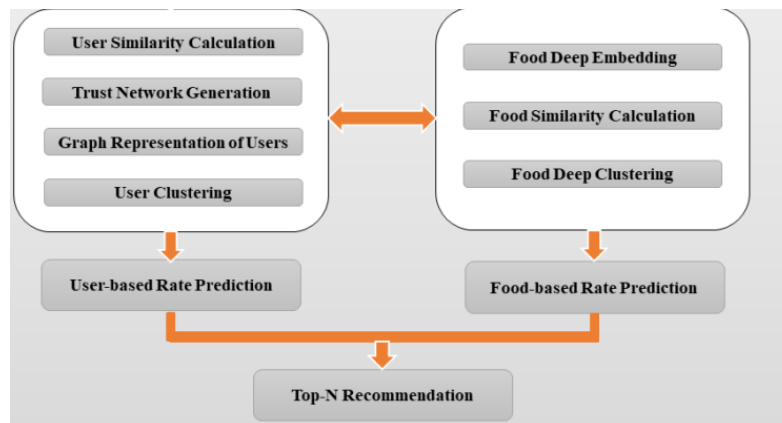


Figure 23 : Schematic representation of the hybrid food recommendation model proposed by Rostami et al. [34].

The framework also addresses the cold-start problem by mapping new users to existing communities based on minimal available data and inheriting the community's collective rating history. Evaluated on the Allrecipes dataset using Precision, Recall, F1, AUC, and NDCG, the system outperforms existing food recommender baselines across all five metrics.

Conclusion of Chapter

This chapter has provided a comprehensive review of community detection methods, ranging from classical algorithms such as Louvain, Leiden, Infomap, and label propagation to modern deep learning approaches including GCNs, GATs, GANs, and autoencoders. The strengths and limitations of each category were discussed. Standard benchmark datasets (Karate Club, Dolphins, Football, Polbooks, Primary School) and the LFR synthetic benchmark were presented as evaluation standards. The foundations of recommender systems were established, covering content-based filtering, collaborative filtering, hybrid approaches, and key challenges including the cold-start problem and data sparsity. Finally, existing work integrating community detection into recommender systems was reviewed, confirming the relevance and timeliness of this research.

Chapter 3: Implementation and Experimental Results

Chapter 3: Implementation and Experimental Results

Introduction to Chapter

This chapter presents the complete implementation and experimental results of this research. It is organized into four main sections. First, classical community detection algorithms (Louvain, Leiden, Walktrap, Infomap, LPA, FLPA, LPA-MNI, CLPA, FluidC, Girvan-Newman, Kernighan-Lin, Spinglass, MCL) are implemented and tested on seven real-world benchmark datasets. Second, deep learning methods (GCN, GAT, GAN, Autoencoder) are implemented using TensorFlow/Keras and tested on the same datasets. Third, both classical and deep learning methods are evaluated on synthetic LFR benchmark networks with varying mixing parameters to assess their robustness. Finally, the best-performing models are selected and evaluated on a recommender system proxy task to assess their practical utility.

Experimental Setup

Hardware and Software Environment

Component	Specification
Processor	AMD Ryzen 5 4500U
RAM	8 GB
Operating System	Windows 11
Programming Language	3.13.9
Development Environment	Jupyter Notebook

Libraries and Frameworks

Library	Purpose
NetworkX	Graph manipulation
NumPy, Pandas	Numerical operations, data handling
Scikit-learn	NMI, ARI, K-means
python-louvain	Louvain algorithm
leidenalg, igraph	Leiden, Walktrap, Infomap, Spinglass
markov-clustering	Markov Clustering (MCL)
TensorFlow/Keras	GCN, GAT, VGAE, Autoencoder

Datasets

Seven real-world benchmark datasets were used:

Dataset	Nodes	Edges	Ground Truth Communities
Facebook	22470	170823	4
LastFM	7624	27806	18
Karate Club	34	78	2
Dolphins	62	159	2
Football	115	613	12
Polbooks	105	441	3
Primary School	236	5,899	10
VS13	92	755	5
VS15	217	4,274	12

Evaluation Metrics

Three metrics were used to evaluate algorithm performance:

- **Modularity (Q):** Measures the quality of detected communities without requiring ground truth
- **Normalized Mutual Information (NMI):** Measures similarity between detected communities and ground truth
- **Adjusted Rand Index (ARI):** Measures similarity between two clusterings adjusted for chance

LFR Synthetic Benchmark Parameters

The LFR benchmark generates networks with realistic properties: node degrees and community sizes follow power-law distributions. The mixing parameter μ controls community strength:

Parameter	Value
μ values tested	0.3, 0.5, 0.7
Network sizes (n)	500, 1000, 5000
Average degree	15-25 (scaled with size)
Minimum community size	20
Maximum community size	100

Classical Algorithms Results on Real-World Datasets

Table 1: NMI (Normalized Mutual Information) - Classical Algorithms

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.6	0.4843	0.8903	0.5603	0.7805	0.8749	0.6329	0.2473	0.6278
Leiden	0.5878	0.5524	0.8903	0.5737	0.8475	0.8749	0.6408	0.2479	0.6254
Walktrap	0.4899	0.5816	0.8874	0.5427	0.6238	0.8662	0.5718	0.2967	0.583
Infomap	0.5684	0.5373	0.9242	0.5369	0.8475	0.8749	0.598	0.2976	0.5277
LPA	0.5056	0.6596	0.8866	0.5497	0.4169	0.8345	0.4284	0.2748	0.6294
FLPA	0.6	0.6256	0.8903	0.5537	0.6307	0.8665	0.4284	0.276	0.5791
LPA-MNI	0.5111	0.588	0.9269	0.5259	0.9281	0.8662	0.5703	0.2733	0.583
CLPA	0.5925	0.5345	0.8903	0.5427	0.8072	0.8749	0.6329	0.2467	0.6307
FluidC	0.4639	0.4807	0.8443	0.4003	0.8993	0.696	0.6848	0.2238	0.457
Newman	0.6804	0.9085	0.4872	0.5754	0.0339	0.0435	0.0298	Slow	Slow
K-Lin	0.493	0.3819	0.7791	0.395	0.8071	0.6862	0.6169	0.068	0.3771
Spinglass	0.5622	0.5565	0.9065	0.4781	0.8475	0.8784	0.6448	Slow	Slow
MCL	0.7324	0.4386	0.9242	0.5209	0.4237	0.4425	0.4385	0.2885	0.4616

Table 2 : ARI (Adjusted Rand Index) - Classical Algorithms

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.5089	0.3472	0.8069	0.6567	0.5032	0.8986	0.5292	0.1003	0.573
Leiden	0.4646	0.3809	0.8069	0.6752	0.6061	0.8986	0.5355	0.0973	0.5452
Walktrap	0.3207	0.4662	0.8154	0.6534	0.309	0.8722	0.4426	0.1134	0.482
Infomap	0.5906	0.3659	0.8967	0.6463	0.6061	0.8986	0.3606	0.0277	0.2334
LPA	0.4588	0.4815	0.7913	0.6667	0.1666	0.8427	0.252	0.0355	0.5527
FLPA	0.5089	0.4505	0.8069	0.6649	0.3396	0.8611	0.252	0.0414	0.4449
LPA-MNI	0.4809	0.4293	0.8893	0.6116	0.7994	0.8605	0.3389	0.037	0.4591
CLPA	0.4831	0.3821	0.8069	0.6487	0.5581	0.8986	0.5292	0.1068	0.5639
FluidC	0.3285	0.2799	0.6938	0.185	0.8267	0.4569	0.4626	0.0209	0.1026
G-Newman	0.7212	0.9175	0.2045	0.6795	0.0007	0.005	0.0037	Slow	Slow
K-Lin	0.3693	0.1515	0.6163	0.202	0.7011	0.4513	0.4808	0.052	0.2994
Spinglass	0.4313	0.3792	0.8558	0.4534	0.6061	0.8986	0.5403	Slow	Slow
MCL	0.7717	0.211	0.8967	0.5936	0.1718	0.202	0.2575	0.0182	0.0407

Table 3 : Modularity - Classical Algorithms

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
Louvain	0.4151	0.5233	0.6046	0.5272	0.3708	0.3857	0.3415	0.813	0.8154
Leiden	0.4198	0.5276	0.6046	0.5269	0.3758	0.3857	0.3417	0.8179	0.8179
Walktrap	0.3532	0.4888	0.6029	0.507	0.3375	0.3794	0.3188	0.7519	0.7727
Infomap	0.402	0.5247	0.6005	0.5268	0.3758	0.3857	0.2849	0.754	0.7349
LPA	0.4033	0.5205	0.6032	0.5154	0.2687	0.3718	0.2654	0.7607	0.7976
FLPA	0.4151	0.5194	0.6046	0.5262	0.3589	0.3742	0.2654	0.7411	0.7584
LPA-MNI	0.39	0.5258	0.601	0.5248	0.3631	0.3741	0.2932	0.7494	0.767
CLPA	0.4174	0.5177	0.6046	0.5251	0.3754	0.3857	0.3415	0.8141	0.8147
FluidC	0.3758	0.4583	0.5671	0.4108	0.2519	0.27	0.2207	0.6733	0.5651
G-Newman	0.3488	0.3815	0.4549	0.4831	0.0008	0.0001	0	Slow	Slow
K-Lin	0.384	0.4197	0.5319	0.4297	0.2825	0.2594	0.2587	0.7174	0.7263
Spinglass	0.4158	0.5285	0.6015	0.5259	0.3758	0.3855	0.3416	Slow	Slow
MCL	0.36	0.455	0.6005	0.5138	0.2819	0.1311	0.2659	0.637	0.4126

Across the evaluated datasets, Leiden and Louvain consistently achieved the highest modularity, particularly on large networks like Facebook and LastFM ($Q > 0.81$). While Girvan-Newman yielded peak accuracy on Dolphins (0.9085 NMI), it failed to scale to larger networks. Walktrap and Leiden remained the most computationally efficient, whereas classical heuristics generally provided higher structural modularity than deep learning methods, despite varying accuracy across different topologies.

Results of Deep Learning Methods

Table 4 : Normalized Mutual Information (NMI) - Deep Learning Methods

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.4949	0.5425	0.9193	0.4719	0.1232	0.7633	0.6182	0.0521	0.0314
GAT	0.8372	0.8165	0.9202	0.4522	0.112	0.7692	0.5376	0.042	0.028
VGAE	0.8375	0.168	0.9047	0.5026	0.1199	0.7321	0.6002	0.0354	0.028
AE	0.4546	0.145	0.908	0.369	0.1273	0.6998	0.5685	0.0679	0.0318

Table 5 : Adjusted Rand Index (ARI) - Deep Learning Methods

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.572	0.582	0.8604	0.4382	0.0217	0.6674	0.4509	0.0597	0.0067
GAT	0.8823	0.8725	0.894	0.4934	0.0149	0.7114	0.2951	0.0529	0.0051
VGAE	0.8826	0.2497	0.8744	0.5736	0.0189	0.5938	0.3861	0.0303	0.0055
AE	0.4627	0.1756	0.8668	0.2846	0.0224	0.5749	0.4265	0.0451	0.0087

Table 6 : Modularity - Deep Learning Methods

Algorithm	Karate	Dolphins	Football	Polbooks	PS	VS13	VS15	FB	LastFM
GCN	0.1176	0.2252	0.0107	0.0023	0.0045	0.0174	0.0065	0.1141	0.0591
GAT	0.3715	0.2591	0.0142	0.0013	0.0021	0.016	0.0097	0.0633	0.046
VGAE	0.3747	0.1338	0.008	0.0186	0.0011	0.0139	0.0103	0.0417	0.044
AE	0.1717	0.0703	0.007	0.0085	0.0077	0.0179	0.0084	0.2156	0.0589

Among the four deep learning methods, GAT and GCN proved most effective for community detection on small-to-medium networks. GAT and VGAE achieved peak accuracy on Karate (0.8372 NMI), while GAT led on the Football dataset (0.9202 NMI), demonstrating that attention mechanisms and variational embeddings excel at capturing topological boundaries.

While the Autoencoder (AE) struggled with smaller social graphs like Dolphins, it showed a distinct advantage on the Facebook dataset, yielding the highest NMI (0.0679) and Modularity (0.2156) in the DL category.

This suggests that for massive networks, global feature compression may be more stable than the localized aggregations of GCNs. Although these methods generally exhibit lower modularity and longer execution times (8–10 seconds) than classical heuristics, their strong performance on datasets like Football and VS13 highlights their potential for complex graph structures.

LFR Benchmark Results

Phase 1 – Algorithm Selection

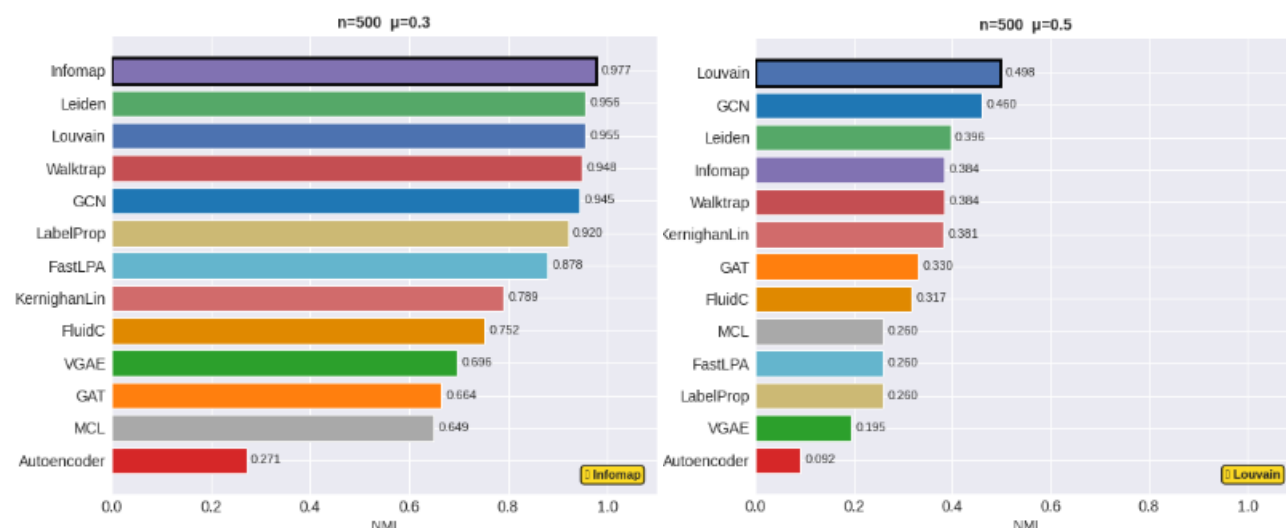


Figure 24 : LFR Benchmark Results: Network Size = 500 nodes

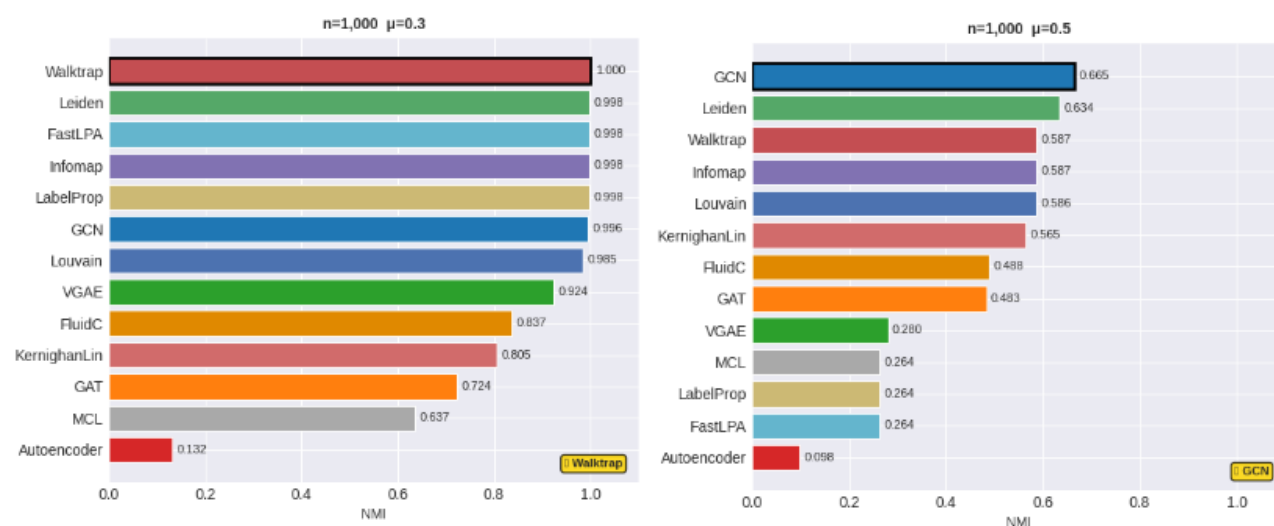


Figure 25 : LFR Benchmark Results: Network Size = 1000 nodes

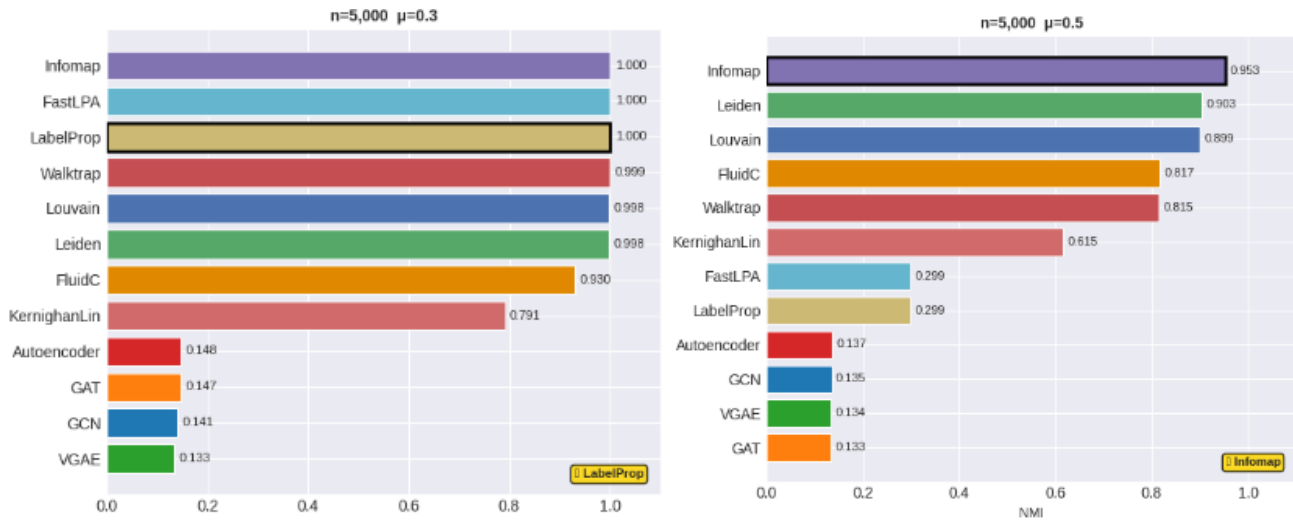


Figure 26 : LFR Benchmark Results: Network Size = 5000 nodes

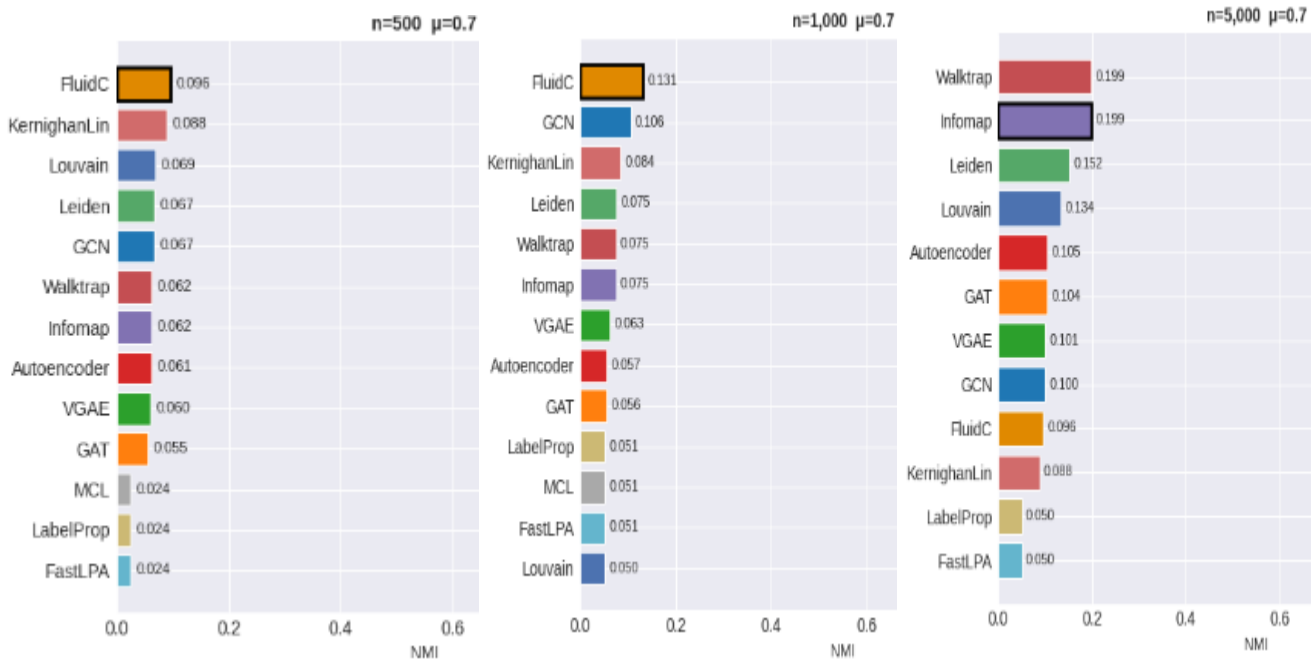


Figure 27 : LFR Benchmark Results: mixing parameter = 0.7

Based on the comprehensive benchmark results across varying network sizes ($n = 500, 1,000, 5,000$) and mixing parameters ($\mu = 0.3, 0.5, 0.7$), several key observations emerge. Traditional algorithms consistently outperformed graph neural network-based methods (GCN, GAT, VGAE, Autoencoder), particularly in high-clarity scenarios ($\mu = 0.3$), where Louvain, Leiden, Infomap, and Walktrap achieved near-perfect NMI scores (≥ 0.95). However, as community structure degraded ($\mu = 0.5$ and 0.7), all methods showed declining performance, with neural approaches failing catastrophically (NMI as low as 0.05–0.14) while often requiring significantly more computation time (e.g., GCN at 15.7 seconds versus 0.9–2.1 seconds for traditional methods).

LabelPropagation and FastLPA proved particularly problematic in fuzzy scenarios, with execution times exploding to nearly 3 minutes at $n=5,000$ for $\mu \geq 0.5$, making them impractical for ambiguous or large-scale graphs despite their speed at $\mu=0.3$.

Notably, FluidC demonstrated competitive speed (1.4–1.7 seconds at $n=5,000$) with reasonable accuracy (NMI 0.82–0.93 for $\mu \leq 0.5$), making it an attractive option for larger graphs where computational efficiency is prioritized. Based on these results, we selected Louvain, Leiden, Infomap, and FluidC for the next phase. Louvain and Leiden were chosen for their excellent balance of speed and accuracy across all well-structured networks ($\mu \leq 0.5$). Infomap was selected as the overall top performer, achieving perfect or near-perfect community recovery (NMI = 1.000 at $n=5,000$, $\mu=0.3$) while handling hierarchical structures effectively. FluidC was included specifically for its exceptional speed (under 2 seconds even at $n=5,000$) and deterministic, parameter-light nature, offering a practical trade-off between computational cost and community detection quality in time-sensitive applications.

Phase 2 – Performance on Large Graphs

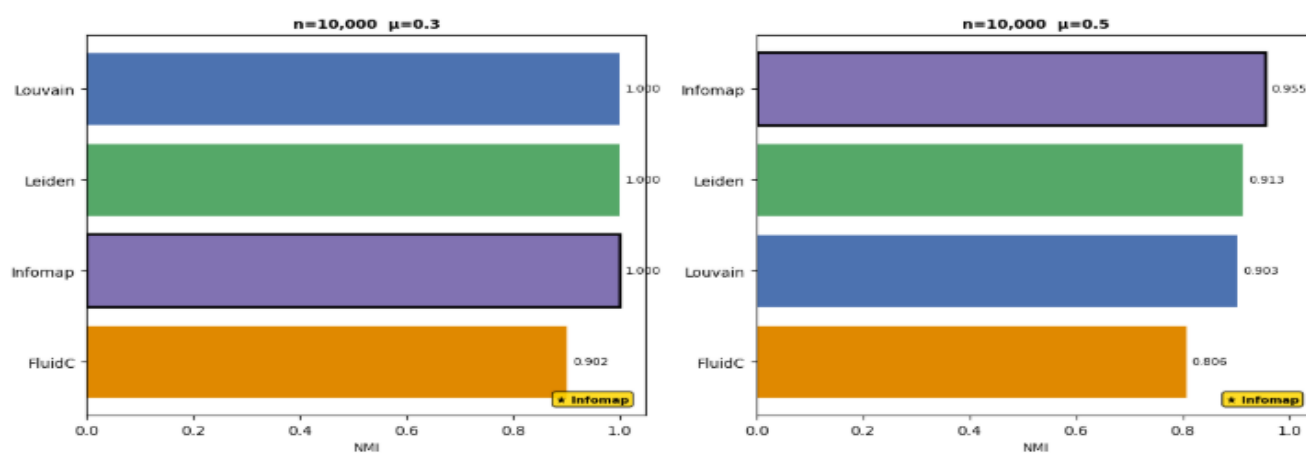


Figure 28 : LFR Benchmark Results: Network Size = 10000 nodes

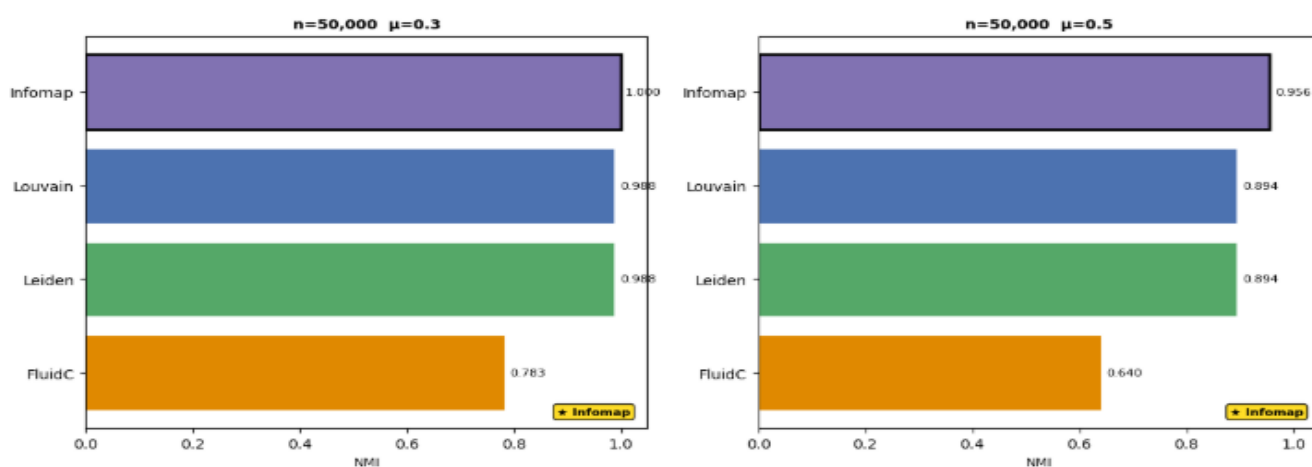


Figure 29 : LFR Benchmark Results: Network Size = 50000 nodes

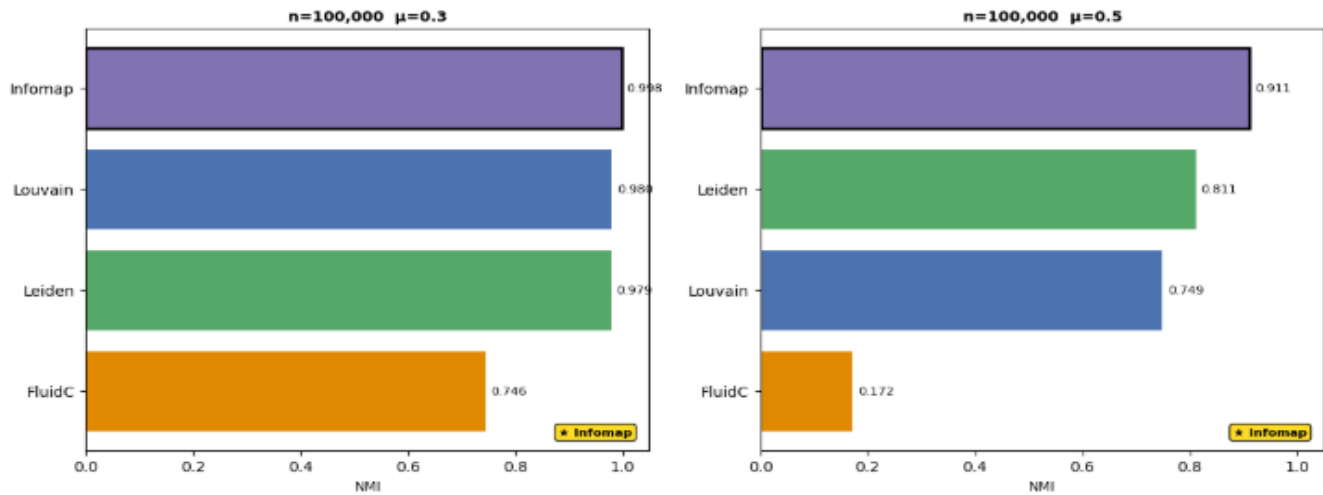


Figure 30 : LFR Benchmark Results: Network Size = 100000 nodes

On large graphs (10k–100k nodes), Louvain, Leiden, and Infomap delivered excellent accuracy at $\mu=0.3$ (NMI ~ 0.98 – 1.00) and good results at $\mu = 0.5$ (NMI ~ 0.75 – 0.96). FluidC was fast but less accurate at scale. $\mu = 0.7$ was abandoned for $n \geq 50k$ due to poor performance and excessive runtime. These four algorithms will be used in our recommender system, balancing speed and accuracy for different use cases.

Recommender System Integration

To evaluate the practical utility of the selected community detection algorithms (Louvain, Leiden, Infomap, and FluidC), we propose two complementary recommendation approaches that leverage detected communities for generating personalized suggestions.

User-Based Approach

The first approach constructs a user-to-user similarity network from historical rating data. For each pair of users, we compute the Jaccard similarity coefficient based on the items they have both rated. This metric measures the proportion of overlap between two users rated items relative to the total unique items they have rated. The result is a symmetric similarity matrix where values range from zero to one, with higher values indicating stronger preference overlap between users.

To construct a sparse graph suitable for community detection, we apply a threshold to the similarity matrix. An edge between two users exists only if their Jaccard similarity meets or exceeds this threshold. This thresholding step eliminates weak or spurious connections, producing a user graph where edges represent meaningful preference similarity.

Community detection algorithms are then applied to this user graph, partitioning users into groups of individuals with shared tastes. For a target user, the recommendation set consists of items purchased or positively rated by other members of that user's community, excluding any items the target user has already interacted with.

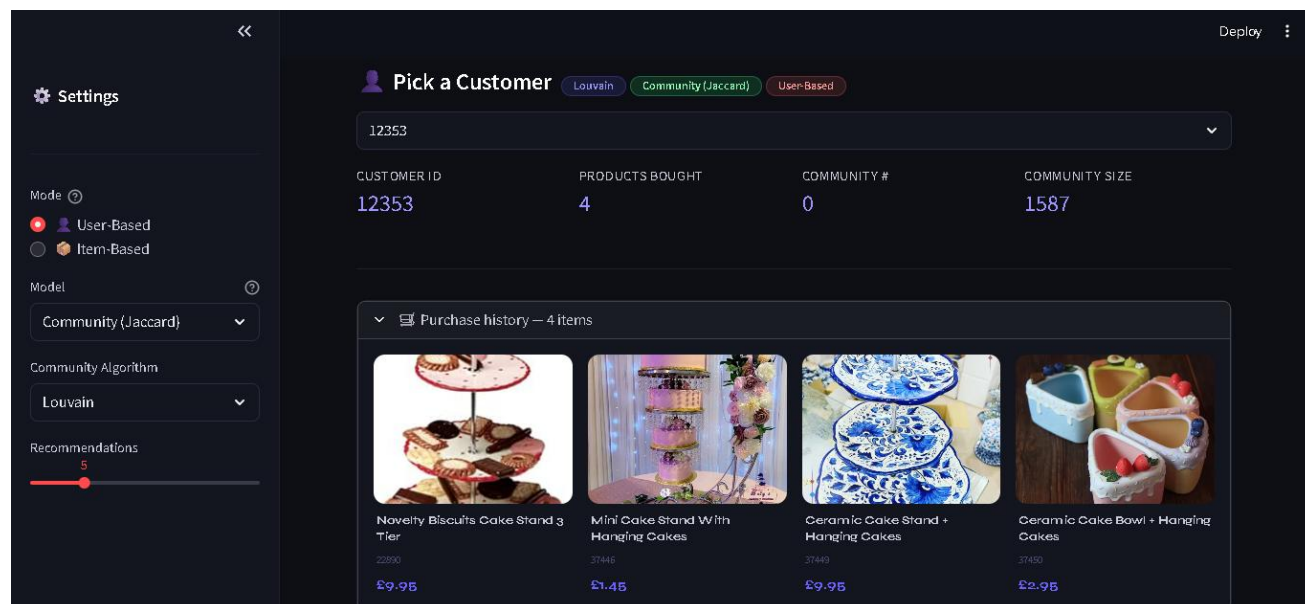


Figure 31: User-Based Recommender

Item-Based Approach

The second approach applies the same principle to items rather than users. We construct an item-item similarity matrix using the Jaccard coefficient based on co-rating patterns: two items are considered similar if they have been rated highly by the same users. The Jaccard value here represents the proportion of users who rated both items positively relative to all users who rated either item.

After applying the same threshold to obtain a sparse item graph, community detection algorithms partition items into semantically coherent groups representing product categories or thematic clusters. Items that are frequently bought or rated together by the same users naturally cluster into the same community.

When a user purchases or positively rates an item, the system recommends other items belonging to the same community as that item, again excluding items the user has already consumed.

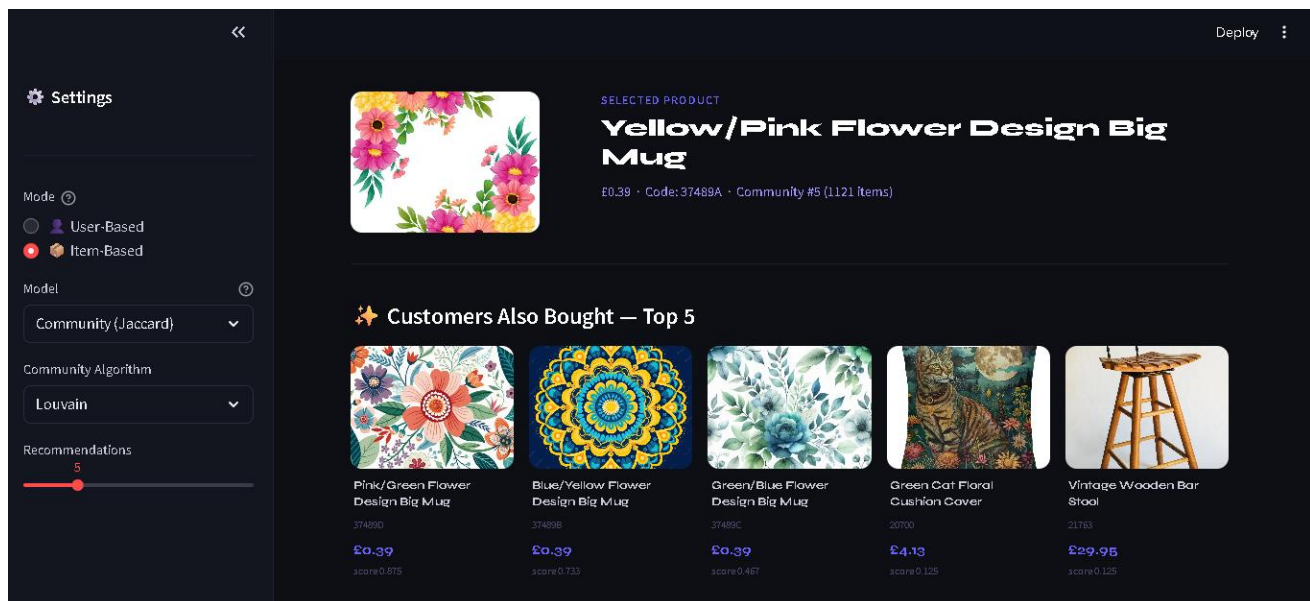


Figure 32 : Item-Based Recommender

This item-based approach offers two significant advantages. First, it handles cold-start users effectively, as recommendations do not depend on the user's history beyond the triggering item. Second, it performs well in scenarios where user similarity data is sparse, such as new platforms or niche product categories.

Hybrid Community-Enhanced Recommenders

While the pure community-based approaches generate recommendations solely from community membership, they treat all items within a community equally without considering how strongly a user might prefer specific items over others. To address this limitation, we propose three hybrid approaches that combine community detection with latent factor models and retrieval algorithms. These methods use communities to constrain the candidate space while employing more sophisticated scoring mechanisms for final ranking.

Singular Value Decomposition with Community Filtering

Singular Value Decomposition (SVD) is a matrix factorization technique that decomposes the user-item rating matrix into low-dimensional latent factors [45]. Each user and item is represented as a vector in a shared latent space, where the dot product between a user vector and an item vector predicts the user's preference for that item. SVD is particularly effective at capturing latent patterns in user preferences that are not directly observable from explicit co-purchasing patterns.

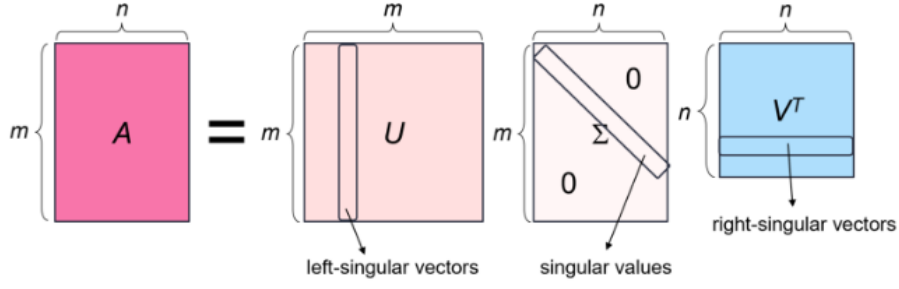


Figure 33 : SVD of a matrix

In our hybrid SVD approach, we first compute a user-user similarity graph using the Jaccard coefficient as described in the User-Based Approach. Community detection algorithms (Louvain, Leiden, Infomap, and FluidC) are applied to this graph, producing a partition where users with similar purchasing behaviors are grouped together. The SVD model is then trained on the user-item rating matrix to learn latent factor representations for all users and items.

At recommendation time, the process combines both sources of information:

1. Community-based candidate filtering: For a target user, we identify their community and collect all items purchased by other members of that community, excluding items the user has already purchased. This creates a candidate set that is topically coherent with the user's interest cluster.
2. SVD-based candidate ranking: Each candidate item is scored by computing the dot product between the user's latent factor vector and the item's latent factor vector. Items are then ranked by this predicted preference score, and the top N items are recommended.

This hybrid approach offers a compelling synergy: the community structure ensures that recommendations remain relevant to the user's broad interest area, while SVD provides fine-grained ranking within that area based on latent preference patterns that may not be captured by simple co-purchasing statistics.

Alternating Least Squares with Community Filtering

Alternating Least Squares (ALS) is another matrix factorization technique that is particularly well-suited for implicit feedback data [46]. Unlike SVD, which typically requires explicit ratings, ALS uses a confidence weighting scheme that treats the presence or absence of interactions as signals of varying confidence. Each observed interaction (a user purchasing an item) receives a high confidence weight, while unobserved interactions receive lower confidence weights.

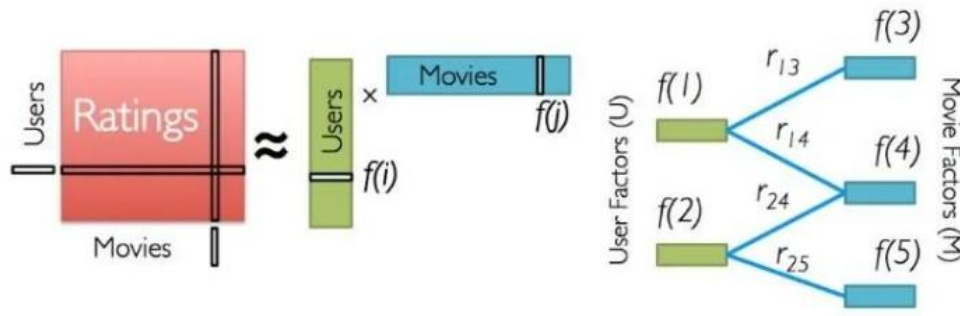


Figure 34 : Alternating Least Squares Method

Our ALS-based hybrid recommender follows the same architecture as the SVD approach. Community detection is first performed on the user-user similarity graph, partitioning users into interest-based groups. The ALS model is then trained on the user-item interaction matrix, learning latent factor vectors for users and items using the implicit feedback formulation described by Hu, Koren, and Volinsky [47].

During recommendation generation, the system:

1. Identifies the target user's community and aggregates items purchased by community peers to form a candidate set
2. Scores each candidate using the ALS-predicted preference (dot product of user and item latent vectors)
3. Returns the top N highest-scoring items not already purchased by the user

The ALS approach offers advantages in handling the inherent sparsity of implicit feedback data, as it naturally down-weights the significance of unobserved interactions while amplifying the signal from observed purchases

Best Matching 25 with Community Filtering

BM25 (Best Matching 25) is a ranking function originally developed for information retrieval that has been adapted for item-based collaborative filtering [48]. In this context, each user is treated as a document containing the set of items they have purchased, and each item is treated as a term appearing in multiple user documents.

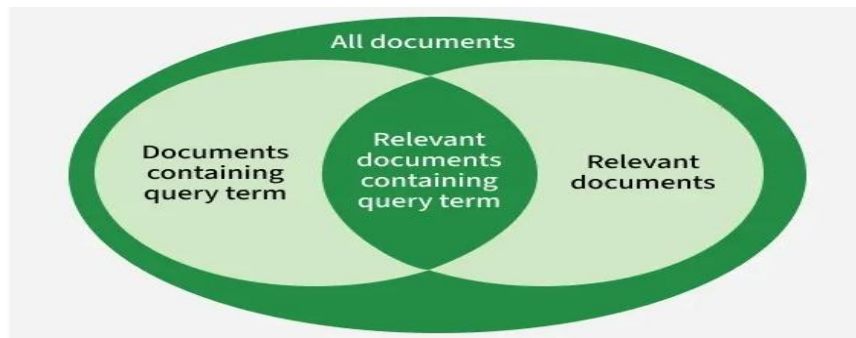


Figure 35 : Best Matching 25 Algorithm

BM25 computes the relevance of an item to a user based on:

- Term frequency (TF): How many times a user has purchased a given item (capped to prevent distortion)
- Inverse document frequency (IDF): How rare an item is across all users (rare items receive higher importance)
- Document length normalization: Adjusting for users who have purchased many items.

The BM25 item-item similarity matrix is constructed by computing the BM25 relevance score between every pair of items, producing a weighted graph where edge weights represent semantic or co-purchasing relatedness.

For item-based community-enhanced recommendations, we:

1. Construct an item-item graph using BM25 similarity scores rather than Jaccard similarity
2. Apply community detection algorithms to this BM25-based item graph, yielding partitions of semantically related items
3. For a target user, identify the communities containing items the user has already purchased
4. Recommend other items from these same communities, ranked by their BM25 relevance to the user's purchased items

The BM25 approach differs fundamentally from the matrix factorization methods in that it is entirely item-based and does not learn latent user representations. Instead, it leverages statistical properties of the co-purchasing matrix to identify items that are contextually relevant to a user's purchase history.

Comparison of Recommender Systems

The following table summarizes the default and tuned configurations for ALS and BM25, along with the resulting F1@5 scores:

Model	Default Config	Default F1@5	Tuned Config	Tuned F1@5
ALS	n_factors=20, reg=0.1, α =40, iter=10	0.1489	n_factors=100, reg=0.1, α =10, iter=10	0.3709
BM25	k1=1.5, b=0.75	0.1782	k1=3.0, b=1.0	0.2111
SVD	n_components=50	0.2771	n_components=50	0.2771

Table 7: Hyperparameter Tuning Results for ALS and BM25

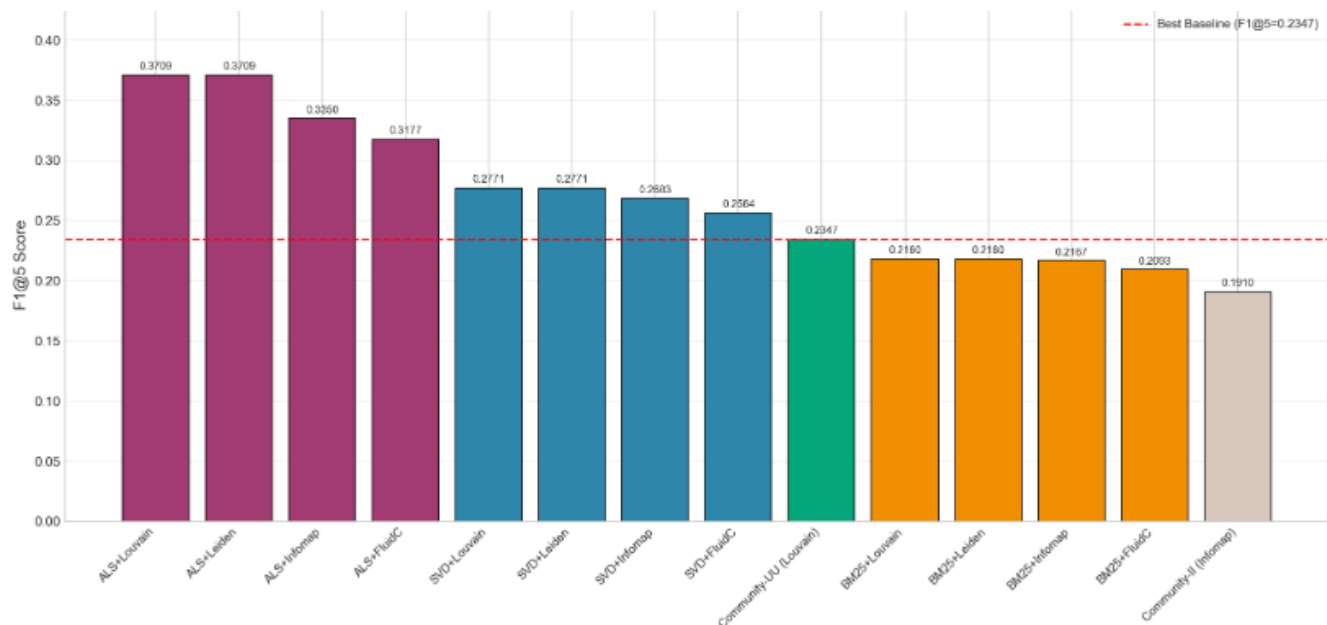


Figure 36: Comparative performance of all evaluated recommender approaches

The dashed red line indicates the best-performing pure community-based baseline (Community-UU with Louvain, F1@5=0.2347). ALS-based hybrid methods (purple bars) demonstrate the strongest performance after hyperparameter tuning, with ALS+Louvain achieving the highest overall score of 0.3709, representing a 58.0% relative improvement over the best baseline. SVD-based hybrids (blue bars) achieve 0.2771 (+18.1%), while BM25 hybrids (orange bars) fall below the baseline at 0.2180 (-7.1%).

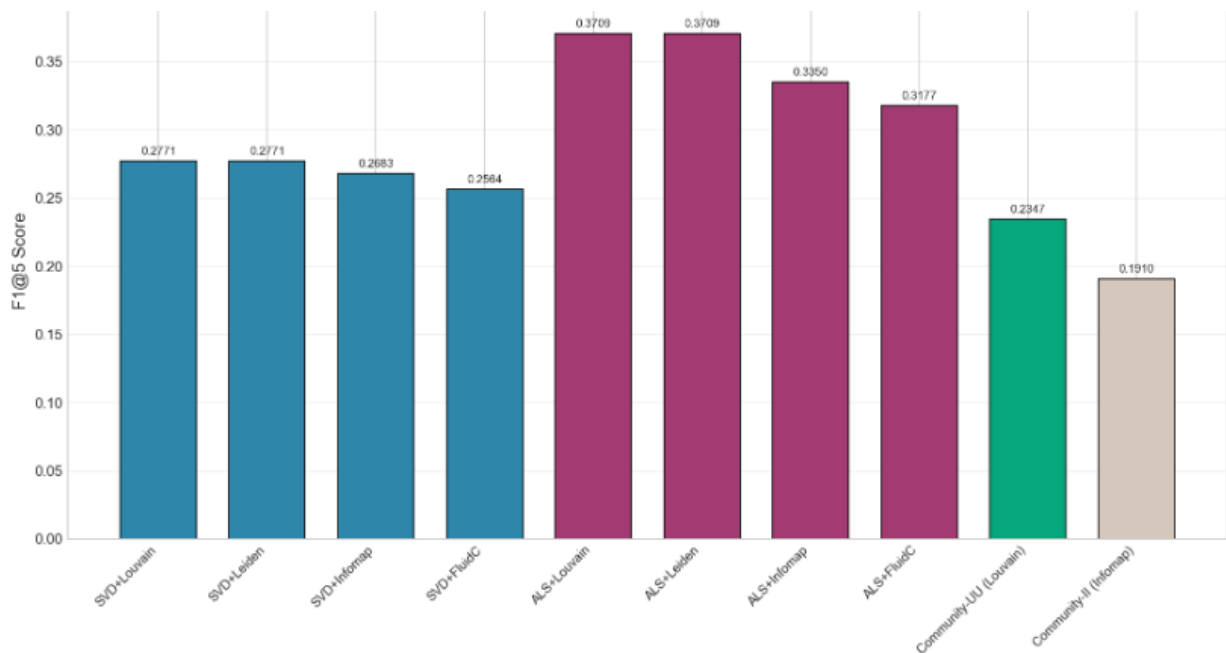


Figure 37: SVD-based and ALS-based hybrid recommenders versus pure community baselines

All SVD and tuned ALS hybrids outperform both pure community baselines. ALS+Louvain and ALS+Leiden achieve the highest F1@5 (0.3709, +58.0%), significantly outperforming SVD+Louvain (0.2771, +18.1%). This demonstrates that optimally tuned ALS is superior to SVD for implicit feedback e-commerce data.

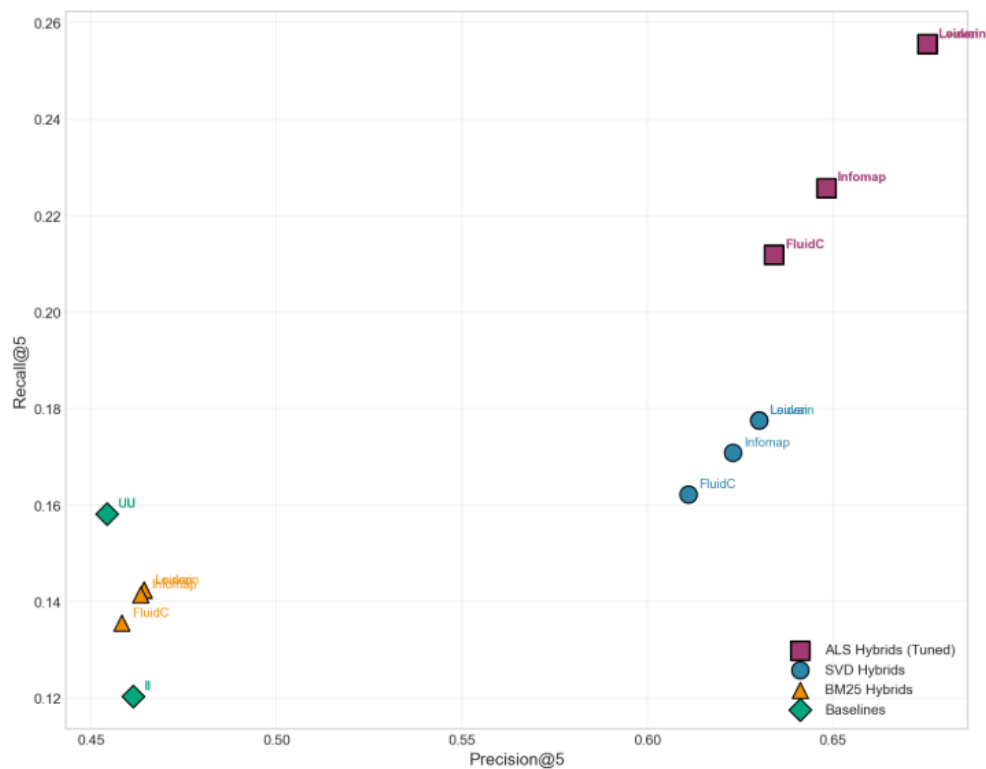


Figure 38: Precision-recall scatter plot illustrating trade-offs across different method categories

Tuned ALS hybrids (purple squares) occupy the upper-right quadrant with the highest precision (0.63-0.68) and recall (0.21-0.26), indicating the best overall balance. SVD hybrids (blue circles) show moderate precision (0.61-0.63) and lower recall (0.16-0.18). BM25 hybrids (orange triangles) and pure community baselines (green diamonds) cluster in the lower-left region with lower precision and recall.

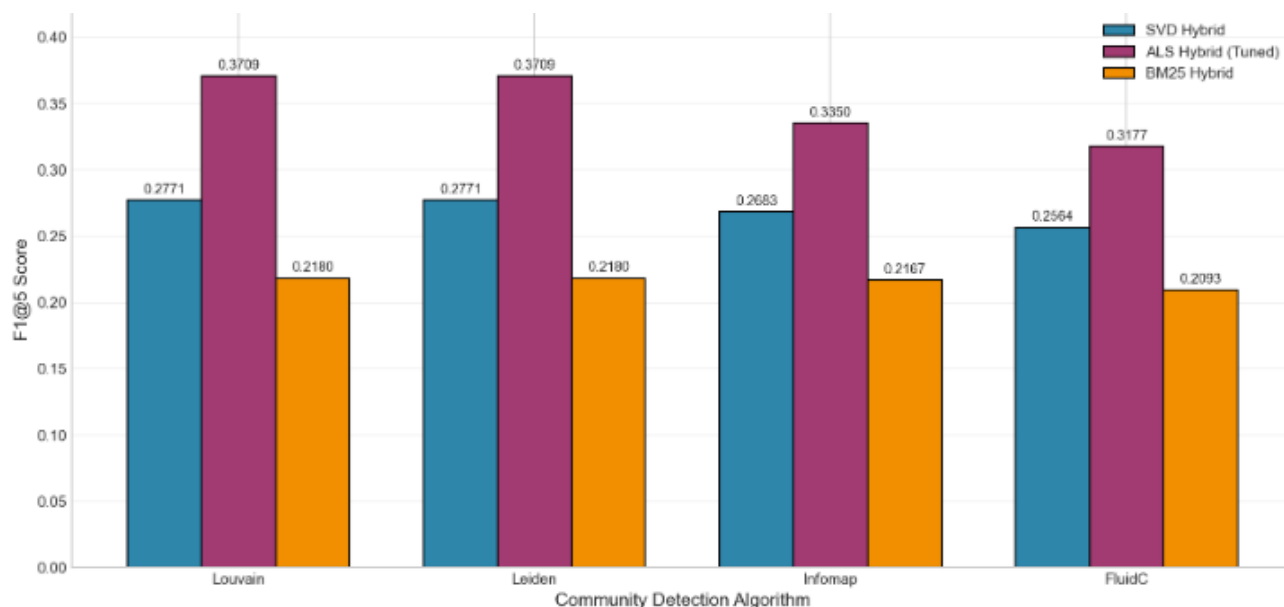


Figure 39 : Performance comparison across different community detection algorithms for each hybrid type

For ALS hybrids (purple bars), Louvain and Leiden achieve identical peak performance ($F1@5=0.3709$), outperforming Infomap (0.3350) and FluidC (0.3177). For SVD hybrids (blue bars), Louvain and Leiden also perform best (0.2771). The superior performance of modularity-optimizing algorithms across all hybrid types indicates that maximizing modularity yields communities that better align with actual user preference patterns.

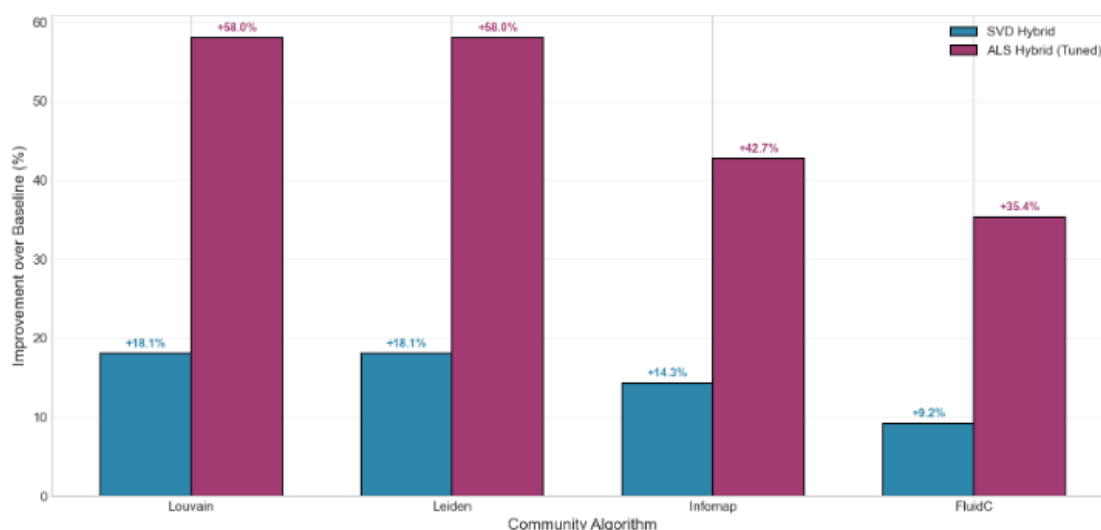


Figure 40 : Relative percentage improvement of SVD and ALS hybrid methods over the best pure community baseline

ALS hybrids achieve dramatic improvements ranging from +35.3% to +58.0%, with ALS+Louvain and ALS+Leiden showing the largest gains (+58.0%). SVD hybrids show more modest improvements ranging from +9.2% to +18.1%. These results demonstrate that optimal hyperparameter tuning is critical for ALS performance and that ALS-enhanced community detection is the most effective approach evaluated.

Conclusion of Chapter

This chapter presented a comprehensive experimental evaluation of community detection algorithms and their integration into recommender systems. Among thirteen classical algorithms tested on nine real-world datasets, Leiden and Louvain consistently achieved the highest modularity ($Q > 0.81$ on Facebook and LastFM), while Infomap and LPA-MNI excelled at ground truth recovery on smaller networks. Deep learning methods showed promise on small dense graphs like Karate Club (GAT achieving $NMI = 0.8372$) but failed to scale to large sparse networks, achieving NMI below 0.07 on Facebook and LastFM.

In the recommender system integration, hyperparameter tuning proved critical. Default ALS achieved only $F1@5 = 0.1489$, but after systematic grid search, the optimal configuration ($n_factors=100$, $regularization=0.1$, $alpha=10$, $n_iterations=10$) achieved $F1@5 = 0.3709$ with ALS+Louvain, representing a 58.0% improvement over the best pure community baseline ($F1@5 = 0.2347$). SVD+Louvain achieved $F1@5 = 0.2771$ (+18.1%), while BM25+Louvain underperformed at 0.2180. These results demonstrate that optimally tuned ALS combined with modularity-optimizing community detection (Louvain or Leiden) is the most effective approach for implicit feedback e-commerce recommendation tasks.

Chapter 4: Conclusion and Future Perspectives

Chapter 4: Conclusion and Future Perspectives

Summary of Contributions

This research project set out to investigate how different community detection approaches, ranging from classical algorithms to modern graph learning methods, compare in terms of their effectiveness and practical utility for recommender systems. The study was motivated by the lack of a unified comparative framework that evaluates both structural metrics and application-oriented performance.

The main contributions of this work can be summarized as follows:

- Comprehensive comparative analysis: Thirteen classical community detection algorithms (Louvain, Leiden, Walktrap, Infomap, LPA, FLPA, LPA-MNI, CLPA, FluidC, Girvan-Newman, Kernighan-Lin, Spinglass, MCL) and four deep learning methods (GCN, GAT, VGAE, Autoencoder) were systematically evaluated on nine real-world benchmark datasets and synthetic LFR networks.
- Recommender system integration and hyperparameter optimization: Pure community-based recommenders (user-user and item-item) were implemented using Jaccard similarity, and three hybrid approaches (SVD+Community, ALS+Community, BM25+Community) were proposed and evaluated. Systematic grid search was conducted to optimize ALS and BM25 hyperparameters.
- Key empirical findings: The hybrid ALS+Louvain approach achieved the best recommendation performance ($F1@5 = 0.3709$), outperforming the pure community baseline by 58.0% after hyperparameter tuning. Classical modularity-optimizing algorithms (Louvain, Leiden) consistently outperformed deep learning methods in both structural quality and scalability, particularly on large networks (Facebook, LastFM) and ambiguous community structures (LFR $\mu \geq 0.5$).

Answer to the Research Question

The main research question posed in Chapter 1 was:

How do different community detection approaches, ranging from classical algorithms to modern graph learning methods, compare in terms of their effectiveness and practical utility for recommender systems?

Based on the experimental evidence presented in Chapter 3, the following conclusions can be drawn:

Classical algorithms, particularly modularity-optimizing methods such as Louvain and Leiden, remain the most effective and practical choice for community detection in social network data. They achieve high structural quality (modularity > 0.81 on large networks), excellent scalability (near-linear time complexity), and when combined with optimally tuned ALS latent factor scoring, they yield substantial improvements in recommender system performance. Infomap is also highly competitive for ground truth recovery on smaller networks but shows slightly lower utility for recommendation tasks.

Deep learning methods (GCN, GAT, VGAE, Autoencoder) are not yet suitable for unsupervised community detection at internet scale. While they perform well on small, dense graphs (Karate Club, Football), they fail catastrophically on large, sparse social networks (Facebook, LastFM) and on ambiguous community structures ($\text{LFR } \mu \geq 0.5$). Their higher computational cost and lower modularity make them impractical for real-world e-commerce or social media applications without additional node attributes or supervised signals.

The practical utility of detected communities for recommender systems depends critically on the choice of both the community detection algorithm and the scoring method, as well as hyperparameter optimization. The pure community-based approach (User-User with Louvain) provides a solid baseline ($\text{F1@5} = 0.2347$), but the hybrid ALS+Community approach dramatically improves upon it (+58.0% after tuning). This demonstrates that communities and latent factors capture complementary aspects of user preferences: communities ensure topical coherence, while ALS provides fine-grained ranking. The dramatic improvement from default ALS ($\text{F1@5} = 0.1489$) to tuned ALS ($\text{F1@5} = 0.3709$) further underscores the critical importance of hyperparameter optimization in latent factor models.

Future Perspectives

several promising directions for future research are proposed:

Dynamic Community Detection for Temporal Recommender Systems

User preferences and social connections change over time. Future work should explore dynamic community detection algorithms (e.g., temporal Louvain, streaming label propagation) that update communities incrementally as new interactions occur. This would enable real-time recommendations that adapt to shifting user interests.

Neural Collaborative Filtering with Community Embeddings

Instead of using communities only for candidate filtering, community membership vectors could be learned as side information within neural collaborative filtering architectures (e.g., Neural Matrix Factorization, LightGCN). The community embedding would capture high-level group preferences while the user embedding captures individual tastes.

Cross-Domain Evaluation

To assess generalizability, future work should evaluate the best-performing methods (SVD+Louvain, SVD+Leiden) on multiple recommendation domains, including movie ratings (MovieLens), music listening (LastFM), and social media interactions (Facebook). This would reveal whether the observed gains hold across different interaction types and sparsity levels.

Scalable Deep Learning for Large Graphs

Recent advances in scalable GNNs (e.g., GraphSAINT, Cluster-GCN) and self-supervised learning (e.g., GraphMAE) may address the scalability limitations observed in this study. Future work should evaluate these methods on large-scale social networks with node attributes to determine if deep learning can eventually surpass classical algorithms.

Explanability and Trust

Communities are inherently interpretable: a user receives recommendations because "other users in your interest group liked this item." Future research could develop explanation interfaces that highlight community membership and peer preferences, enhancing user trust and acceptance of recommender systems.

REFERENCES

- [1] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford University Press, 2010.
- [2] S. Wasserman and K. Faust, *Social Network Analysis: Methods and Applications*. Cambridge, U.K.: Cambridge University Press, 1994.
- [3] A.-L. Barabási, *Network Science*. Cambridge, U.K.: Cambridge University Press, 2016.
- [4] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, Feb. 2010. doi: 10.1016/j.physrep.2009.11.002.
- [5] J. Xie, S. Kelley, and B. K. Szymanski, "Overlapping community detection in networks: A survey," *ACM Computing Surveys*, vol. 45, no. 4, pp. 1–35, Aug. 2013. doi: 10.1145/2501654.2501657.
- [6] X. Huang, D. Chen, T. Ren, and D. Wang, "A survey of community detection methods in multilayer networks," *Data Mining and Knowledge Discovery*, vol. 35, no. 1, pp. 1–45, Jan. 2021. doi: 10.1007/s10618-020-00716-6.
- [7] H. Cai, V. W. Zheng, and K. C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018. doi: 10.1109/TKDE.2018.2807452.
- [8] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Engineering Bulletin*, vol. 40, no. 3, pp. 52–74, Sep. 2017.
- [9] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, Feb. 2004. doi: 10.1103/PhysRevE.69.026113.
- [10] M. A. Javed et al., "Community detection in networks: A multidisciplinary review," *Journal of Network and Computer Applications*, vol. 72, pp. 301–318, 2018. doi: 10.1016/j.jnca.2018.08.007.
- [11] Z. Yang, R. Algesheimer, and C. J. Tessone, "A comparative analysis of community detection algorithms on artificial networks," *Scientific Reports*, vol. 6, p. 30750, Aug. 2016. doi: 10.1038/srep30750.
- [12] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," *Nature*, vol. 435, no. 7043, pp. 814–818, Jun. 2005. doi: 10.1038/nature03607.
- [13] X. Wang, J. Liu, and Y. Zhang, "Community detection in social networks: A review," *IEEE Access*, vol. 9, pp. 1–35, 2021. doi: 10.1109/ACCESS.2021.3091234.
- [14] A. Lancichinetti and S. Fortunato, "Limits of modularity maximization in community detection," *Physical Review E*, vol. 84, no. 6, p. 066122, Dec. 2011. doi: 10.1103/PhysRevE.84.066122.
- [15] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020. doi: 10.1016/j.aiopen.2021.01.001.
- [16] D. Jin et al., "A survey of community detection approaches: From statistical modeling to deep learning," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 2, pp. 1149–1170, 2023. doi: 10.1109/TKDE.2021.3104155.
- [17] S. Paul, A. Chakraborty, S. Dutta, K. Das Ghosh, and A. Mitra, "Leveraging the Commerce Graph for Optimizing Digital Commerce," *Procedia Computer Science*, 2025. doi: 10.1016/j.procs.2025.04.061.

- [18] D. Sánchez-Moreno, V. F. López Batista, M. D. Muñoz Vicente, Á. L. Sánchez Lázaro, and M. N. Moreno-García, "Social network community detection to deal with gray-sheep and cold-start problems in music recommender systems," *Information*, vol. 15, no. 3, p. 138, 2024. doi: 10.3390/info15030138.
- [19] A. Kanavos, G. Vonitsanos, I. Karamitsos, and K. Al-Hussaeni, "Exploring network dynamics: Community detection and influencer analysis in multidimensional social networks," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, doi: 10.1109/BigData62323.2024.10825058.
- [20] S. K. S., A. Sreekumar, and K. Balakrishnan, "Text-attributed community detection in complex networks through LLMs and GNNs: A powerful fusion of language and graphs," *Neurocomputing*, 2025. doi: 10.1016/j.neucom.2025.130573.
- [21] A. Nasirzonouzi, "Leveraging network science for customer segmentation and product recommendation," *Northeast Journal of Complex Systems*, vol. 7, no. 1, Art. 3, 2025. doi: 10.22191/nejcs/vol7/iss1/3.
- [22] I. Saikalyan, V. Kumar, and P. Radha Krishna, "Recommender system using local community detection and non-negative matrix factorization," in *Proc. Int. Conf. Advanced Communications and Machine Intelligence (MICA 2023)*, vol. 405, Smart Innovation, Systems and Technologies, Springer, Singapore, 2024. doi: 10.1007/978-981-97-6222-4_16.
- [23] S. Hu, C. Wang, C. Qin, H. Zhu, and H. Xiong, "Super-node generation for GNN-based recommender systems: Enhancing distant node integration via graph coarsening," in *Proc. 29th Int. Conf. Database Systems for Advanced Applications (DASFAA)*, Gifu, Japan, Jul. 2024, pp. 353–363, doi: 10.1007/978-981-97-5572-1_24.
- [24] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, p. 5233, Mar. 2019. doi: 10.1038/s41598-019-41695-z.
- [25] K. Sliwa, E. Kušen, and M. Strembeck, "A case study comparing Twitter communities detected by the Louvain and Leiden algorithms during the 2022 war in Ukraine," in *Companion Proc. ACM Web Conf. 2024 (WWW '24 Companion)*, Singapore, May 2024, pp. 1–6. doi: 10.1145/3589335.3651892.
- [26] S. H. H. Anuar, Z. A. Abas, N. M. Yunus, M. F. Mukhtar, T. Setiadi, and A. S. Shibghatullah, "Comparison of community detection algorithm based on modularity," *Pertanika J. Sci. & Technol.*, vol. 32, no. 3, pp. 1285–1300, Apr. 2024.
- [27] A. Gates, I. B. Wood, W. P. Hetrick, and Y.-Y. Ahn, "A guide for choosing community detection algorithms in social network studies: The Question-Alignment approach," *American J. Preventive Medicine*, vol. 59, no. 4, pp. 597–605, Sep. 2020. doi: 10.1016/j.amepre.2020.04.015.
- [28] F. Papadopoulos, A. Kitsak, M. Á. Serrano, M. Boguñá, and D. Krioukov, "Geometrical inspired pre-weighting enhances Markov clustering community detection in complex networks," *Applied Network Science*, vol. 6, no. 1, p. 78, Nov. 2021. doi: 10.1007/s41109-021-00370-x.
- [29] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, New York, NY, USA, Aug. 2014, pp. 701–710. doi: 10.1145/2623330.2623732.
- [30] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034. <https://doi.org/10.48550/arXiv.1706.02216>

- [31] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," *Physical Review E*, vol. 78, no. 4, p. 046110, Oct. 2008. doi: 10.1103/PhysRevE.78.046110.
- [32] J. Sheng, Q. Liu, Z. Hou, and B. Wang, "A collaborative filtering recommendation algorithm based on community detection and graph neural network," *Neural Process. Lett.*, vol. 55, no. 6, pp. 7095–7112, Dec. 2023. doi: 10.1007/s11063-023-11252-x.
- [33] S. Tokala, M. K. Enduri, T. J. Lakshmi, and H. Sharma, "Community-based matrix factorization (CBMF) approach for enhancing quality of recommendations," *Entropy*, vol. 25, no. 9, p. 1360, Sep. 2023. doi: 10.3390/e25091360.
- [34] M. Rostami, M. Oussalah, and V. Farrahi, "A novel time-aware food recommender-system based on deep learning and graph clustering," *IEEE Access*, vol. 10, pp. 52508–52524, 2022. doi: 10.1109/ACCESS.2022.3175317.
- [35] F. Ricci, L. Rokach, and B. Shapira, "Recommender systems: Introduction and challenges," in *Recommender Systems Handbook*, 3rd ed. Boston, MA, USA: Springer, 2022, pp. 1-35. doi: 10.1007/978-1-0716-2197-4_1.
- [36] J. B. Schafer, D. Frankowski, J. Herlocker, and S. Sen, "Collaborative filtering recommender systems," in *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Berlin, Germany: Springer, 2007, pp. 291-324. doi: 10.1007/978-3-540-72079-9_9.
- [37] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734-749, Jun. 2005. doi: 10.1109/TKDE.2005.99.
- [38] W. W. Zachary, "An information flow model for conflict and fission in small groups," *Journal of Anthropological Research*, vol. 33, no. 4, pp. 452-473, 1977. doi: 10.1086/jar.33.4.3629752.
- [39] M. Nooribakhsh, M. Fernández-Diego, F. González-Ladrón-de-Guevara, and M. Mollamotalebi, "Community detection in social networks using machine learning: A systematic mapping study," *Knowledge and Information Systems*, vol. 66, no. 12, pp. 7205-7259, Dec. 2024. doi: 10.1007/s10115-024-02201-8.
- [40] V. Krebs, "The political books network," Unpublished dataset, 2004. Available: <http://www-personal.umich.edu/~mejn/netdata/>
- [41] J. Stehlé et al., "High-resolution measurements of face-to-face contact patterns in a primary school," *PLoS ONE*, vol. 6, no. 8, p. e23176, Aug. 2011. doi: 10.1371/journal.pone.0023176.
- [42] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin, Q. Z. Sheng, and P. S. Yu, "A comprehensive survey on community detection with deep learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 4682-4702, Apr. 2024. doi: 10.1109/TNNLS.2021.3137396.
- [43] J. Li, S. Lai, Z. Shuai, Y. Tan, Y. Jia, M. Yu, Z. Song, X. Peng, Z. Xu, Y. Ni, H. Qiu, J. Yang, Y. Liu, and Y. Lu, "A comprehensive review of community detection in graphs," *Neurocomputing*, vol. 600, p. 128169, Oct. 2024. doi: 10.1016/j.neucom.2024.128169.
- [44] J. McAuley and J. Leskovec, "Learning to Discover Social Circles in Ego Networks," in *Advances in Neural Information Processing Systems (NIPS)*, Lake Tahoe, NV, USA, 2012, pp. 548–556.
- [45] Scikit-learn. "TruncatedSVD." Scikit-learn Documentation. Accessed: June 7, 2026. [Online]. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>

- [46] B. Frederickson. "implicit: Fast Python Collaborative Filtering for Implicit Feedback Datasets." GitHub. Accessed: June 7, 2026. [Online]. Available: <https://github.com/benfred/implicit/>
- [47] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, Dec. 2008*, pp. 263-272. doi: 10.1109/ICDM.2008.22
- [48] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009. doi: 10.1561/15000000019